

Computing IV Sec 201: Project Portfolio

Lillyanna Yim

Spring 2025

Contents

1	PS0: Hello SFML	2
2	PS1: PhotoMagic	6
3	PS2: Triangle Fractal	12
4	PS3: Dynamic N-Body Simulation	17
5	PS4: Sokoban	27
6	PS5: DNA String Alignment	37
7	PS6: RandWriter	42
8	PS7: Kronos Log Parsing	52

Time to Complete Portfolio: 27 hours

1 PS0: Hello SFML

1.1 Discussion

Once the program is opened, it shows a yellow circle and a dog egg that moves according to keystrokes; specifically the arrow keys. If you hold down the 'A' key, the dog egg will start to follow your mouse instead. I thought it would be more interesting to let the user have multiple ways to move the sprite, depending on what felt more natural to them. I also changed the circle color from green to yellow because it reminded me of the sun and felt more cheerful. For the sprite, I chose a dog egg because I thought it was really fun and interesting to look at. At first, I had bigger plans, like adding music, but I struggled to find the right file path to get it working. I ended up dropping the idea and instead focused on refining the movement options. Even though it was not my original plan, I still think the final project turned out creative and unique.

1.2 What I accomplished

I was able to make a working SFML window that displays a moving circle and sprite. The sprite can move based on the arrow keys, or if you hold down the 'A' key, it will start following your mouse. I also added the ability to switch between the two movement types while the program is running, which makes it feel a lot more interactive. On top of that, I made small design choices like changing the color of the circle to yellow and picking a fun sprite to keep things more exciting.

1.3 What I already knew

Before starting this project, I already knew some basic programming concepts like how to set up simple if-statements and loops. I also had a general understanding of how event handling works, like how key presses can control what happens in a window. I was familiar with the idea of sprites, but I had not worked with them much in an actual graphics library like SFML yet.

1.4 What I learned

Through this project, I learned how to actually use SFML to make shapes and load a sprite onto the screen. I figured out how to track the mouse's position and make a sprite move toward it in real time, which was something I thought was really unique. I also learned how to make the sprite movement switch depending on what key you are holding down, and how to make the program responsive in different ways without crashing. Working with the SFML documentation also gave me a better idea of how to find answers when I get stuck.

1.5 Challenges

One of the biggest challenges was trying to add sound. I was really excited to add a music feature because I had already picked out the song, but I couldn't get the file path to work. After spending a lot of time trying to fix it, I had to accept that it was not going to happen for this project. I started brainstorming other extra features, but a lot of my ideas did not fit the assignment guidelines. Eventually, I came up with the idea to let the sprite switch between arrow key movement and mouse following, and even though it wasn't what I originally planned, I'm really happy with how it turned out.

1.6 Proof of Execution

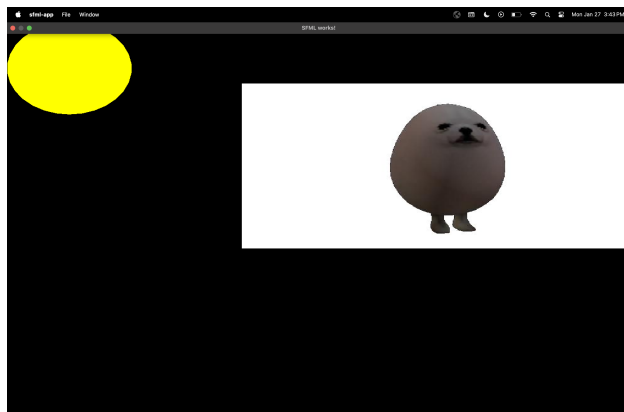


Figure 1: Screenshot of running program

1.7 Code

```
1 CC = g++
2 CFLAGS = --std=c++20 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -
    lboost_unit_test_framework
4 # Your .hpp files
5 DEPS =
6 # Your compiled .o files
7 OBJECTS =
8 # The name of your program
9 PROGRAM = sfml-app
10
11 .PHONY: all clean lint
12
13
14 all: $(PROGRAM)
15
16 # Wildcard recipe to make .o files from corresponding .cpp file
17 %.o: %.cpp $(DEPS)
18     $(CC) $(CFLAGS) -c $<
19
20 $(PROGRAM): main.o $(OBJECTS)
21     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
22
23 clean:
24     rm *.o $(PROGRAM)
25
26 lint:
27     cpplint *.cpp *.hpp
```

```
1 // copyright 2025 Lillyanna Yim
2
3 #include <SFML/Graphics.hpp>
4
5 // follow mouse
6 void follow(sf::RenderWindow& window, sf::Sprite &sprite) {
7     // gets the position of the mouse relative to the window
8     sf::Vector2i mousePosition = sf::Mouse::getPosition(window);
9
10    // convert window relative position to global position
11    sf::Vector2f wholePosition = window.mapPixelToCoords(
12        mousePosition);
```

```

13         sprite.setPosition(wholePosition); // sets the position of the
           sprite to the mouse
14     }
15
16 int main() {
17     sf::RenderWindow window(sf::VideoMode(1000, 1000), "SFML works!");
18     sf::CircleShape shape(100.f);
19
20     shape.setFillColor(sf::Color::Yellow);
21
22     // creating a sprite (egg dog)
23     sf::Texture texture;
24     if (!texture.loadFromFile("sprite.png"))
25         return EXIT_FAILURE;
26
27     sf::Sprite sprite(texture);
28
29     // sets the sprite to its origin when opening the window
30     sprite.setOrigin(100, -100);
31
32
33
34
35     while (window.isOpen()) {
36         sf::Event event;
37         while (window.pollEvent(event)) {
38             if (event.type == sf::Event::Closed)
39                 window.close();
40         }
41         // move with keystrokes Arrows
42         if (sf::Keyboard::isKeyPressed(sf::Keyboard::Left))
43
44             sprite.move(-1, 0);
45
46         if (sf::Keyboard::isKeyPressed(sf::Keyboard::Right))
47
48             sprite.move(1, 0);
49
50         if (sf::Keyboard::isKeyPressed(sf::Keyboard::Up))
51
52             sprite.move(0, -1);
53
54         if (sf::Keyboard::isKeyPressed(sf::Keyboard::Down))
55
56             sprite.move(0, 1);
57
58         if (sf::Keyboard::isKeyPressed(sf::Keyboard::A))
59
60             // if 'A' is held, it will move with mouse.
61             // if A is not held, then it will move ONLY with keyboard.
62             follow(window, sprite);
63
64         window.clear();
65         window.draw(sprite);
66         window.draw(shape);
67         window.display();
68     }
69
70     return 0;

```


2 PS1: PhotoMagic

2.1 Discussion

This project was focused on implementing a Linear Feedback Shift Register (LFSR) and using it to apply a visual transformation to an image. The LFSR starts with a binary seed and uses bit shifting and feedback logic to create pseudo-random numbers. I used this LFSR in two parts: the first part was to make sure it worked correctly by itself, and the second part was to apply it to each pixel in an image using XOR operations. The final result is an image that looks encrypted. I thought it was interesting how a relatively simple pattern like an LFSR could be used to make something that looks totally scrambled but can still be reversed.

2.2 What I accomplished

I got the LFSR working using a `std::vector<int>` to represent the bits. I implemented the step function to handle one round of feedback and shifting, and the generate function to run multiple steps and output a full number. Once that part worked, I used the LFSR to go through an image pixel-by-pixel and XOR each color value, which gave me an encrypted image effect. It was cool seeing how the same logic that generated bits could be used to transform actual images.

2.3 What I already knew

Going into this project, I already understood basic C++ syntax and how vectors work. I also knew about binary numbers and how XOR works in theory. I had written unit tests before, and I knew how to break down a problem into helper functions, like separating step and generate. But I hadn't really applied that to an image-processing task before.

2.4 What I learned

I learned how to design and test a working LFSR from scratch, and I got a better understanding of how bit-level operations can be used in real applications like image encryption. I learned how to think in terms of binary shifts and how to use XOR in a way that actually changes visual data. I also learned how to structure a C++ program that involves both logic and graphics, and how to test each part so I could be confident that it was working the way I expected.

2.5 Challenges

One challenge was understanding how the bits were supposed to shift and making sure I was calculating the feedback bit correctly. Sometimes, even one wrong index would throw off the whole result. Another tricky part was figuring out how to apply the LFSR to the image without messing up the original data. I had to make sure I was applying the XOR operation properly to each color value without affecting transparency or other pixel properties. Debugging was a little tough at times, especially when the image didn't look quite right and I wasn't sure which part of the code was the problem.

2.6 Proof of Execution

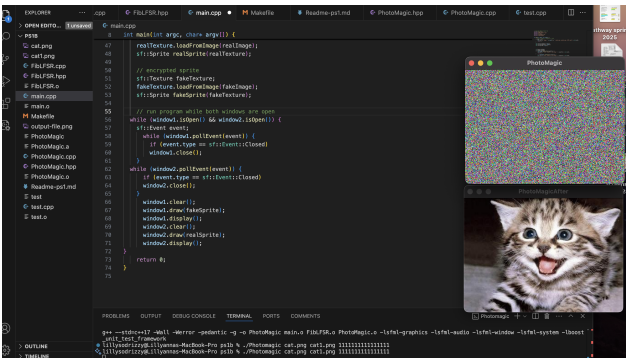


Figure 2: Screenshot of running program

2.7 Code

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -
    lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = FibLFSR.hpp PhotoMagic.hpp
6 # Your compiled .o files
7 OBJECTS = FibLFSR.o PhotoMagic.o
8 # The name of your program
9 PROGRAM = FibLFSR
10
11 .PHONY: all clean lint
12
13 all: test PhotoMagic PhotoMagic.a
14
15 %.o: %.cpp $(DEPS)
16     $(CC) $(CFLAGS) -c $<
17
18 $(PROGRAM): main.o $(OBJECTS)
19     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
20
21 PhotoMagic: main.o $(OBJECTS)
22     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
23
24 test: test.o $(OBJECTS)
25     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
26
27
28 PhotoMagic.a: $(OBJECTS)
29     ar rcs $@ $^
30
31 clean:
32     rm *.o $(PROGRAM) test
33
34 lint:
35     cpplint *.cpp *.hpp
36
37 // Copyright 2025 Lillyanna Yim
38 #include <iostream>
39 #include <string>
40 #include <SFML/Graphics.hpp>
41 #include "PhotoMagic.hpp"
42 #include "FibLFSR.hpp"
```

```

7
8 int main(int argc, char* argv[]) {
9     // check if number of arguments is right
10    if (argc != 4) {
11        std::cout << "Usage: " << argv[0] <<
12        " <input.png> <output.png> <LFSR seed>" << std::endl;
13        return 1;
14    }
15
16    std::string inputFile = argv[1];
17    std::string outputFile = argv[2];
18    std::string seed = argv[3];
19
20    sf::Image realImage;
21    sf::Image fakeImage;
22    if (!realImage.loadFromFile(inputFile)) {
23        std::cout << "Error: Unable to load image " << std::endl;
24        return 1;
25    }
26    if (!fakeImage.loadFromFile(inputFile)) {
27        std::cout << "Error: Unable to load image " << std::endl;
28        return 1;
29    }
30
31    // create window
32    sf::Vector2u size = realImage.getSize();
33    sf::RenderWindow window1(sf::VideoMode(size.x, size.y), "PhotoMagic");
34    sf::RenderWindow window2(sf::VideoMode(size.x, size.y), "PhotoMagicAfter
35    ");
36
37    // create LFSR
38    PhotoMagic::FibLFSR lfsr(argv[3]);
39
40    // transform image
41    PhotoMagic::transform(fakeImage, &lfsr);
42    fakeImage.saveToFile(outputFile);
43
44    // original sprite
45    sf::Texture realTexture;
46    realTexture.loadFromImage(realImage);
47    sf::Sprite realSprite(realTexture);
48
49    // encrypted sprite
50    sf::Texture fakeTexture;
51    fakeTexture.loadFromImage(fakeImage);
52    sf::Sprite fakeSprite(fakeTexture);
53
54    // run program while both windows are open
55    while (window1.isOpen() && window2.isOpen()) {
56        sf::Event event;
57        while (window1.pollEvent(event)) {
58            if (event.type == sf::Event::Closed)
59                window1.close();
60        }
61        while (window2.pollEvent(event)) {
62            if (event.type == sf::Event::Closed)
63                window2.close();
64        }

```



```

65     window1.clear();
66     window1.draw(fakeSprite);
67     window1.display();
68     window2.clear();
69     window2.draw(realSprite);
70     window2.display();
71 }
72     return 0;
73 }

```

```

1  // Copyright 2025 Lillyanna Yim
2  #pragma once
3
4  #include <iostream>
5  #include <string>
6  #include <vector>
7
8  namespace PhotoMagic {
9  class FibLFSR {
10 public:
11     explicit FibLFSR(const std::string& seed);
12     explicit FibLFSR(unsigned int seed);
13
14     static FibLFSR fromPassword(const std::string& password);
15
16     int step();
17     int generate(int k);
18 private:
19     std::vector<int> bitContain;
20
21     friend std::ostream& operator<<(std::ostream&, const FibLFSR& lfsr);
22 };
23 } // namespace PhotoMagic

```

```

1  // Copyright 2025 Lillyanna Yim
2  #include <iostream>
3  #include <string>
4  #include "FibLFSR.hpp"
5
6  namespace PhotoMagic {
7  FibLFSR::FibLFSR(const std::string& seed) {
8      // check for empty seed
9      if (seed.size() < 1) {
10         throw std::invalid_argument("invalid argument");
11     }
12     // check for correct seed
13     for (char i : seed) {
14         if (i == '0') {
15             bitContain.push_back(0);
16         }
17         if (i == '1') {
18             bitContain.push_back(1);
19         }
20         if (i != '0' && i != '1') {
21             throw std::invalid_argument("invalid argument");
22         }
23     }
24 }
25
26 // step function

```

```

27 int FibLFSR::step(void) {
28     int start = bitContain[0] ^ bitContain[2] ^ bitContain[3] ^ bitContain
        [5];
29     bitContain.erase(bitContain.begin());
30     bitContain.push_back(start);
31     return start;
32 }
33
34 // generate k bits each time and check if k is valid
35 int FibLFSR::generate(int k) {
36     int result = 0;
37     if (k < 0) {
38         throw std::invalid_argument("invalid argument");
39     }
40     for (int i = 0; i < k; ++i) {
41         result = (result << 1) ^ step();
42     }
43     return result;
44 }
45 std::ostream& operator<<(std::ostream& os, const FibLFSR& lfsr) {
46     for (int bit : lfsr.bitContain) {
47         os << bit;
48     }
49     return os;
50 }
51 } // namespace PhotoMagic

```

```

1 // Copyright 2025 Lillyanna Yim
2 #pragma once
3 #include <algorithm>
4 #include <SFML/Graphics.hpp>
5 #include "FibLFSR.hpp"
6
7 namespace PhotoMagic {
8 void transform(sf::Image& img, FibLFSR* lfsr);
9
10 }

```

```

1 // Copyright 2025 Lillyanna Yim
2 #include "PhotoMagic.hpp"
3 #include <algorithm>
4
5 namespace PhotoMagic {
6 void transform(sf::Image& img, FibLFSR* lfsr) {
7     unsigned int width = img.getSize().x;
8     unsigned int height = img.getSize().y;
9
10    // range base loop to each pixel in the image
11    for (unsigned int y = 0; y < height; y++) {
12        for (unsigned int x = 0; x < width; x++) {
13            // get the color of the pixel at (x, y)
14            sf::Color color = img.getPixel(x, y);
15
16            // get R, G, B, A values of pixel
17            unsigned char r = color.r;
18            unsigned char g = color.g;
19            unsigned char b = color.b;
20            unsigned char a = color.a;
21
22            // XOR, R, G, B values with LFSR

```

```
23         r = r ^ lfsr->generate(8);
24         g = g ^ lfsr->generate(8);
25         b = b ^ lfsr->generate(8);
26
27         // make new color of pixel
28         img.setPixel(x, y, sf::Color(r, g, b, a));
29     }
30 }
31 }
32 } // namespace PhotoMagic
```

3 PS2: Triangle Fractal

3.1 Discussion

This project was about creating a triangle fractal using classes, recursion, and SFML for graphics. The goal was to build the fractal shape by calling functions recursively instead of trying to do it all manually or iteratively. We had to use a combination of object-oriented programming skills, SFML graphics functions, and mathematical calculations to get everything working. This project used what we were currently learning in class and how we can apply our knowledge into this project. Although this was somewhat difficult, it was rewarding once we finally got the fractal to appear correctly on the screen.

3.2 What I accomplished

We built the fractal by creating a class that managed the drawing process and we added helper functions to keep everything organized. Originally we thought about overloading functions to handle different parts of the fractal, but using a class ended up being the cleaner and smarter way to do it. We used recursion to draw the smaller triangles within the larger ones, which made the whole fractal generation process feel almost automatic once we got the math right. It was satisfying to see the pattern appear after working through all the math and setup.

3.3 What I already knew

Before starting we already had a good understanding of basic object-oriented programming in C++; including how to make and use classes. We also knew how recursion worked in theory and we were comfortable using SFML to draw shapes like triangles and lines. However, we hadn't really combined all of those skills together before on a project this big, and we definitely hadn't done much work with fractals or recursive graphics.

3.4 What I learned

I learned a lot about how important careful planning and math are when working with fractals. Coding the functions wasn't actually the hardest part—it was understanding how to move the points around and calculate the positions for the smaller triangles that really challenged us. We had to dive deep into concepts like midpoints and coordinate geometry, and we learned that trial and error is a huge part of getting something visual like this to work. Watching YouTube videos and re-watching lectures became part of the learning process.

3.5 Challenges

The biggest challenge by far was the math behind the fractal. Neither of us felt super confident in geometry, so figuring out how to calculate the correct points for the recursive calls was a big challenge to overcome. We spent a lot of time second-guessing ourselves and adjusting formulas before things started to make sense. It took a lot of patience and persistence, but eventually after testing and troubleshooting, we got it right. It was a real relief when everything finally clicked and the fractal started drawing correctly.

3.6 Proof of Execution

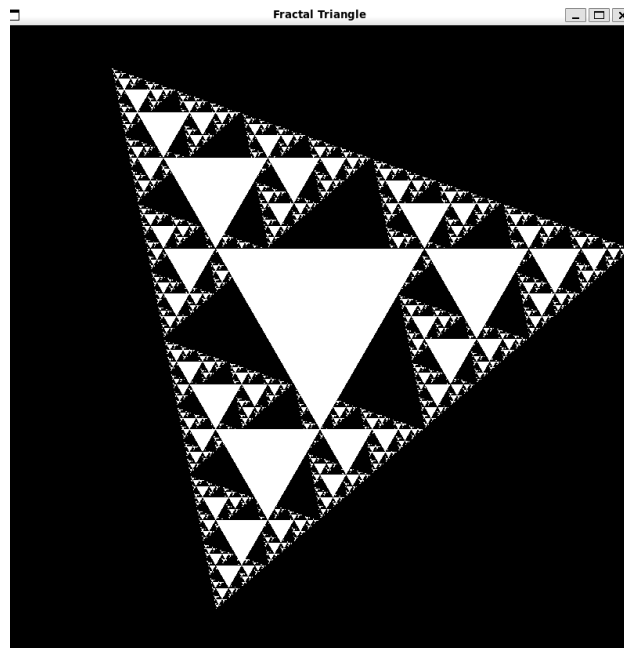


Figure 3: Screenshot of running program

3.7 Code

```
1 CC = g++
2 CFLAGS = -std=c++11 -Wall -Wextra -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system
4 # Your .hpp files
5 DEPS = triangle.hpp
6 # Your compiled .o files
7 OBJECTS = triangle.o main.o
8 # The name of your program
9 PROGRAM = Triangle
10
11 .PHONY: all clean lint
12
13 all: Triangle
14
15 %.o: %.cpp $(DEPS)
16     $(CC) $(CFLAGS) -c $<
17
18 $(PROGRAM): main.o $(OBJECTS)
19     $(CC) $(CFLAGS) -o $@ $~ $(LIB)
20
21 clean:
22     rm -f *.o $(PROGRAM)
23
24 lint:
25     cpplint *.cpp *.hpp
```

```
1 // Copyright Vanzel Essien & Lillyanna Yim 2025
2 #include <iostream>
3 #include <cmath>
4 #include <SFML/Graphics.hpp>
5 #include "triangle.hpp"
6
7 int main(int argc, char* argv[]) {
8     // If the Usage of ./Triangle isnt correct
9     if (argc != 3) {
```

```

10     std::cout << "Usage: " << argv[0] << " <length>" << " <depth>" <<
std::endl;
11     return 1;
12 }
13 // turns the input length string to int
14 int length = std::stoi(argv[1]);
15
16 // turns the input depth string to int
17 int depth = std::stoi(argv[2]);
18 sf::RenderWindow window(sf::VideoMode(length, length), "Fractal Triangle
");
19 float size = length / 3.0f;
20 for (int i = 0; i < depth; i++) {
21     length *= 1.2;
22 }
23 // gets the center of the window
24 sf::Vector2f center(window.getSize().x / 2, window.getSize().y / 2);
25 // creates the fractal in the center
26 Fractal fractal(&window, center, size, depth);
27
28 while (window.isOpen()) {
29     sf::Event event;
30     while (window.pollEvent(event)) {
31         if (event.type == sf::Event::Closed)
32             window.close();
33     }
34
35     window.clear();
36     fractal.draw();
37     window.display();
38 }
39
40 return 0;
41 }

```

```

1 // Copyright Vanzel Essien & Lillyanna Yim 2025
2 #pragma once
3
4 #include <SFML/Graphics.hpp>
5
6 // fractal class
7 class Fractal {
8 public:
9     // Constructor that takes in a window, a point, a side length, and a
depth
10     Fractal(sf::RenderTarget *window, sf::Vector2f point,
11             float side, int depth);
12     // function to draw intital triangle
13     void draw();
14
15 private:
16     // Recursive function to draw the fractal
17     void drawFractal(sf::RenderTarget *window, sf::Vector2f bottom,
18                     sf::Vector2f left, sf::Vector2f right, int depth);
19
20     // variables
21     sf::RenderTarget *window;
22     sf::Vector2f point;
23     float side;
24     int depth;

```

25 };

```
1 // Copyright Vanzel Essien & Lillyanna Yim 2025
2 #include <iostream>
3 #include <cmath>
4 #include "triangle.hpp"
5
6 // Constructor that takes in a window
7 // and point, a side length, and a depth
8 Fractal::Fractal(sf::RenderTarget *window, sf::Vector2f point,
9     float side, int depth) : window(window), point(point),
10     side(side), depth(depth) {}
11
12 // Calculates and draws the intial triangle
13 void Fractal::draw() {
14     float height = side * sqrt(3) / 2;
15     sf::Vector2f left = {-side / 2, -height / 2};
16     sf::Vector2f right = {+side / 2, -height / 2};
17     sf::Vector2f bottom = {0, +height / 2};
18     drawFractal(window, bottom + point, left + point, right + point,
19     depth);
20 }
21 // Recursive function to draw the fractal
22 void Fractal::drawFractal(sf::RenderTarget *window, sf::Vector2f bottom,
23     sf::Vector2f left, sf::Vector2f right, int depth) {
24     if (depth <= 0) {
25         sf::ConvexShape triangle(3);
26         triangle.setFillColor(sf::Color::White);
27         triangle.setPoint(0, bottom);
28         triangle.setPoint(1, left);
29         triangle.setPoint(2, right);
30
31         window->draw(triangle);
32     } else {
33         // gets smaller triangles side length
34         float side = sqrt(pow(left.x - right.x, 2) +
35             pow(left.y - right.y, 2)) / 2;
36         float height = side * sqrt(3) / 2;
37
38         // calculates left triangle points
39         sf::Vector2f lefttrileft = {left.x - (side / 2), left.y - height};
40         sf::Vector2f lefttriright = {left.x + (side / 2), left.y - height};
41         sf::Vector2f lefttribottom = {left.x, left.y};
42
43         // calculates right triangle points
44         sf::Vector2f righttrileft = {right.x, right.y};
45         sf::Vector2f righttriright = {right.x + side, right.y};
46         sf::Vector2f righttribottom = {right.x + (side / 2), right.y +
47         height};
48
49         // calculates bottom triangle points
50         sf::Vector2f bottomtrileft = {bottom.x - side, bottom.y};
51         sf::Vector2f bottomtriright = {bottom.x, bottom.y};
52         sf::Vector2f bottomtribottom = {bottom.x - (side / 2),
53         bottom.y + height};
54
55         sf::ConvexShape triangle(3);
56         triangle.setPoint(0, bottom);
57         triangle.setPoint(1, left);
```

```
57     triangle.setPoint(2, right);
58     window->draw(triangle);
59
60     // recursively calls draw fractal, to draw smaller triangles
61     drawFractal(window, bottom, left, right, depth - 1);
62     drawFractal(window, lefttribottom, lefttrileft,
63         lefttriright, depth - 1);
64     drawFractal(window, righttribottom, righttrileft,
65         righttriright, depth - 1);
66     drawFractal(window, bottomtribottom, bottomtrileft,
67         bottomtriright, depth - 1);
68 }
69 }
```


4 PS3: Dynamic N-Body Simulation

4.1 Discussion

This project simulates the motion of celestial bodies under the force of gravity. Each planet is represented by a `CelestialBody` object, which stores its position, velocity, mass, and image data. These bodies are all managed and updated by the `Universe` class inside the `NB` namespace. The simulation reads initial data from a text file and uses gravitational physics to update each planet's position in real time using SFML. A big part of this project involved reading and writing data from files, so I also overloaded the input and output stream operators for both `Universe` and `CelestialBody` to make testing and data loading simpler. I liked how everything came together visually, and it was cool seeing the planets move in response to physics we coded ourselves.

4.2 What I accomplished

I created a simulation where multiple planets orbit based on real time gravitational calculations. I structured everything around two main classes: `CelestialBody` and `Universe`, both wrapped inside the `NB` namespace. I also implemented the draw function so that everything could be rendered with a single call, which made the program feel neat and manageable. To avoid memory issues, I used smart pointers to manage textures dynamically, especially since each planet needed its own image. For extra polish, I added a real-time elapsed simulation timer using a Calibri font, which made the project feel more interactive and complete.

4.3 What I already knew

Before starting this project, I already had some experience with SFML from earlier assignments. I knew the basics of OOP in C++. I was familiar with how to structure classes and how to overload operators. I also had a general understanding of gravitational physics, so I wasn't starting from scratch, but this was definitely the most complex use of those concepts I'd worked on so far.

4.4 What I learned

I learned how to tie together physics, graphics, and file input/output all in one project. Managing object lifetimes using smart pointers became a lot clearer to me during this assignment, especially when dealing with dynamically loaded textures. I also learned how to transform simulation coordinates into screen coordinates so that the physics calculations and visual rendering matched up. On top of that, I got better at writing clean, testable code by separating concerns between the `Universe` and `CelestialBody` classes.

4.5 Challenges

At first, none of the planets showed up in the window even though I was pretty sure my logic was correct. It turned out my scaling math was off, and once I corrected that things started falling into place. Some early versions of my code also failed a few unit tests because of slight coordinate mismatches. In PS3B, I noticed a few planets still moved too quickly or drifted slightly out of frame, but once I adjusted the view and increased the window size, those issues were fixed. Getting everything aligned visually with the physics was definitely the hardest part, but also the most rewarding.

4.6 Proof of Execution

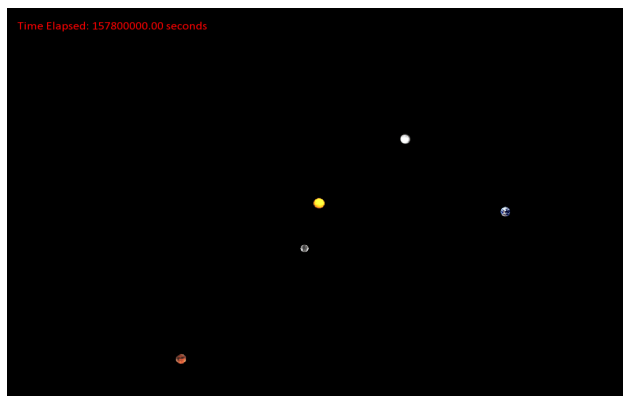


Figure 4: Screenshot of running program

4.7 Code

```
1 CC = g++
2 CFLAGS = --std=c++20 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio -
    lboost_unit_test_framework
4
5 all: NBody NBody.a test
6
7 %.o: %.cpp Universe.hpp CelestialBody.hpp
8     $(CC) $(CFLAGS) -c $<
9
10 NBody: main.cpp Universe.o CelestialBody.o
11     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
12
13 NBody.a: Universe.o CelestialBody.o
14     ar rcs $@ $^
15
16 test: test.cpp Universe.o CelestialBody.o
17     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
18
19 lint:
20     cpplint *.cpp *.hpp
21
22 clean:
23     rm -f NBody NBody.a test *.o
```

```
1 // Copyright 2025 Lillyanna Yim
2 #include <SFML/Graphics.hpp>
3 #include <SFML/Graphics/Text.hpp>
4 #include <SFML/Graphics/Font.hpp>
5 #include <iostream>
6 #include <sstream>
7 #include <string>
8 #include "CelestialBody.hpp"
9 #include "Universe.hpp"
10
11 int main(int argc, char* argv[]) {
12     // Check for correct number of arguments
13     if (argc != 3) {
14         std::cerr << "Usage: " << argv[0] <<
15             " <total time> <time step>" << std::endl;
16         return 1;
17     }
```

```

18
19 // Initialize simulation parameters
20 double totalTime = std::stod(argv[1]);
21 double timeStep = std::stod(argv[2]);
22 double elapsed = 0.0;
23
24 // Create and load universe
25 NB::Universe universe;
26 std::cin >> universe;
27
28 // Create SFML window
29 sf::RenderWindow window(sf::VideoMode(1200, 800), "N-Body Simulation");
30
31 // Setup window center and transform
32 sf::Vector2u windowSize = window.getSize();
33 sf::Vector2f windowCenter(windowSize.x / 2.0f, windowSize.y / 2.0f);
34
35 sf::RenderStates states;
36 states.transform.translate(windowCenter);
37
38 // Setup view to match universe scale
39 float viewSize = 2 * universe.getRadius();
40 sf::View view;
41 view.setSize(viewSize, -viewSize);
42 view.setCenter(0, 0);
43 window.setView(view);
44
45 // Load font for elapsed time text
46 sf::Font font;
47 if (!font.loadFromFile("calibri-regular.ttf")) {
48     std::cerr << "Error: Could not load font!" << std::endl;
49     return 1;
50 }
51
52 // Setup elapsed time text
53 sf::Text timeText;
54 timeText.setFont(font);
55 timeText.setCharacterSize(24);
56 timeText.setFillColor(sf::Color::Red);
57 timeText.setPosition(20.f, 20.f);
58
59 // Main event loop
60 while (window.isOpen()) {
61     sf::Event event;
62     while (window.pollEvent(event)) {
63         if (event.type == sf::Event::Closed) {
64             window.close();
65         }
66     }
67
68     window.clear();
69
70     // Step the universe if time remains
71     if (elapsed < totalTime) {
72         universe.step(timeStep);
73         elapsed += timeStep;
74     }
75
76     // Update bodies' positions

```

```

77     universe.updateBodies();
78
79     // Update elapsed time string
80     std::ostringstream ss;
81     ss.precision(2);
82     ss << std::fixed << "Time Elapsed: " << elapsed << " seconds";
83     timeText.setString(ss.str());
84
85     // Draw universe with simulation view
86     window.setView(view);
87     window.draw(universe, states);
88
89     // Draw elapsed time text with default view
90     window.setView(window.getDefaultView());
91     window.draw(timeText);
92
93     window.display();
94 }
95
96 // Output final universe state
97 std::cout << universe;
98
99 return 0;
100 }

```

```

1 // Copyright 2025 Lillyanna Yim
2 #pragma once
3
4 #include <iostream>
5 #include <memory>
6 #include <string>
7 #include <SFML/Graphics.hpp>
8
9 namespace NB {
10
11 class CelestialBody : public sf::Drawable {
12 public:
13     CelestialBody();
14     explicit CelestialBody(const std::string& imageFile);
15
16     // Accessors
17     sf::Vector2f getPosition() const;
18     sf::Vector2f getVelocity() const;
19     double getMass() const;
20     std::string getFilename() const;
21     double getUniverseRadius() const;
22
23     // Mutators
24     void setPosition(const sf::Vector2f& pos);
25     void setVelocity(const sf::Vector2f& vel);
26     void setMass(double m);
27     void setFilename(const std::string& file);
28     void setUniverseRadius(double radius);
29
30     // Graphics
31     void loadTexture();
32     void initializeSprite();
33     void syncSpritePosition();
34
35     // Draw

```

```

36     void draw(sf::RenderTarget& target, sf::RenderStates states) const
        override;
37
38 private:
39     sf::Vector2f position_;
40     sf::Vector2f velocity_;
41     double mass_;
42     std::string filename_;
43     double universeRadius_;
44     sf::Sprite sprite_;
45     std::shared_ptr<sf::Texture> texturePtr_;
46 };
47
48 std::istream& operator>>(std::istream& input, CelestialBody& body);
49 std::ostream& operator<<(std::ostream& output, const CelestialBody& body);
50
51 } // namespace NB

```

```

1 // Copyright 2025 Lillyanna Yim
2 #include "CelestialBody.hpp"
3 #include <iostream>
4 #include <sstream>
5 #include <string>
6
7 namespace NB {
8
9 CelestialBody::CelestialBody()
10 : position_(0.f, 0.f),
11 velocity_(0.f, 0.f),
12 mass_(0.0),
13 filename_(""),
14 universeRadius_(0.0),
15 sprite_(),
16 texturePtr_(std::make_shared<sf::Texture>()) {}
17
18 CelestialBody::CelestialBody(const std::string& imageFile)
19 : position_(0.f, 0.f),
20 velocity_(0.f, 0.f),
21 mass_(0.0),
22 filename_(imageFile),
23 universeRadius_(0.0),
24 sprite_(),
25 texturePtr_(std::make_shared<sf::Texture>()) {
26 loadTexture();
27 initializeSprite();
28 }
29
30 // Accessors
31 sf::Vector2f CelestialBody::getPosition() const {
32     return position_;
33 }
34
35 sf::Vector2f CelestialBody::getVelocity() const {
36     return velocity_;
37 }
38
39 double CelestialBody::getMass() const {
40     return mass_;
41 }
42

```

```

43 std::string CelestialBody::getFilename() const {
44     return filename_;
45 }
46
47 double CelestialBody::getUniverseRadius() const {
48     return universeRadius_;
49 }
50
51 // Mutators
52 void CelestialBody::setPosition(const sf::Vector2f& pos) {
53     position_ = pos;
54 }
55
56 void CelestialBody::setVelocity(const sf::Vector2f& vel) {
57     velocity_ = vel;
58 }
59
60 void CelestialBody::setMass(double m) {
61     mass_ = m;
62 }
63
64 void CelestialBody::setFilename(const std::string& file) {
65     filename_ = file;
66 }
67
68 void CelestialBody::setUniverseRadius(double radius) {
69     universeRadius_ = radius;
70 }
71
72 // Sprite and Texture Setup
73 void CelestialBody::loadTexture() {
74     if (!texturePtr_->loadFromFile(filename_)) {
75         std::cerr << "Error: Unable to load texture from "
76         << filename_ << std::endl;
77     }
78 }
79
80 void CelestialBody::initializeSprite() {
81     sprite_.setTexture(*texturePtr_);
82     sprite_.setOrigin(texturePtr_->getSize().x / 2.f,
83     texturePtr_->getSize().y / 2.f);
84     sprite_.setScale(universeRadius_ * 2 / 1200.f,
85     universeRadius_ * 2 / 800.f);
86     sprite_.setPosition(position_);
87 }
88
89 void CelestialBody::syncSpritePosition() {
90     sprite_.setPosition(position_);
91 }
92
93 // Drawing
94 void CelestialBody::draw(sf::RenderTarget& target,
95 sf::RenderStates states) const {
96     target.draw(sprite_, states);
97 }
98
99 // Stream Operators
100 std::istream& operator>>(std::istream& input, CelestialBody& body) {
101     float px, py, vx, vy, mass;

```

```

102     std::string file;
103
104     input >> px >> py >> vx >> vy >> mass >> file;
105     body.setPosition(sf::Vector2f(px, py));
106     body.setVelocity(sf::Vector2f(vx, vy));
107     body.setMass(mass);
108     body.setFilename(file);
109     body.loadTexture();
110     body.initializeSprite();
111
112     return input;
113 }
114
115 std::ostream& operator<<(std::ostream& output, const CelestialBody& body) {
116     output << body.getPosition().x << " " << body.getPosition().y << " "
117         << body.getVelocity().x << " " << body.getVelocity().y << " "
118         << body.getMass() << " " <<
119         body.getFilename() << std::endl;
120     return output;
121 }
122
123 } // namespace NB

```

```

1 // Copyright 2025 Lillyanna Yim
2 #pragma once
3
4 #include <iostream>
5 #include <deque>
6 #include <memory>
7 #include <SFML/Graphics.hpp>
8 #include <string>
9 #include "CelestialBody.hpp"
10
11 namespace NB {
12
13 class Universe : public sf::Drawable {
14 public:
15     Universe();
16
17     // Accessors
18     size_t getSize() const;
19     double getRadius() const;
20     double getElapsedTime() const;
21     std::deque<std::shared_ptr<CelestialBody>> getBodies() const;
22
23     // Mutators
24     void setRadius(double r);
25     void setSize(double s);
26     void addBody(std::shared_ptr<CelestialBody> body);
27     void addElapsedTime(double dt);
28
29     // Simulation and Updates
30     void step(double dt);
31     void updateBodies();
32
33     // Operator Overloads
34     const CelestialBody& operator[](size_t index) const;
35
36 protected:
37     // SFML Draw override

```

```

38     void draw(sf::RenderTarget& target, sf::RenderStates states) const
        override;
39
40     private:
41         std::deque<std::shared_ptr<CelestialBody>> bodies_;
42         double radius_ = 0.0;
43         double size_ = 0.0;
44         double elapsedTime_ = 0.0;
45 };
46
47 // Stream Operators
48 std::istream& operator>>(std::istream& input, Universe& universe);
49 std::ostream& operator<<(std::ostream& output, const Universe& universe);
50
51 } // namespace NB

```

```

1 // Copyright 2025 Lillyanna Yim
2 #include <cmath>
3 #include <vector>
4 #include <deque>
5 #include <memory>
6 #include "Universe.hpp"
7
8 namespace NB {
9
10 Universe::Universe() : radius_(0.0), size_(0.0), elapsedTime_(0.0) {}
11
12 // Accessors
13 size_t Universe::getSize() const {
14     return size_;
15 }
16
17 double Universe::getRadius() const {
18     return radius_;
19 }
20
21 double Universe::getElapsedTime() const {
22     return elapsedTime_;
23 }
24
25 std::deque<std::shared_ptr<CelestialBody>> Universe::getBodies() const {
26     return bodies_;
27 }
28
29 // Mutators
30 void Universe::setRadius(double r) {
31     radius_ = r;
32 }
33
34 void Universe::setSize(double s) {
35     size_ = s;
36 }
37
38 void Universe::addBody(std::shared_ptr<CelestialBody> body) {
39     bodies_.push_back(body);
40 }
41
42 void Universe::addElapsedTime(double dt) {
43     elapsedTime_ += dt;
44 }

```



```

45
46 // Simulation step
47 void Universe::step(double dt) {
48     const double G = 6.67e-11; // gravitational constant
49
50     std::vector<double> netForceX(bodies_.size(), 0.0);
51     std::vector<double> netForceY(bodies_.size(), 0.0);
52
53     // Calculate net forces
54     for (size_t i = 0; i < bodies_.size(); ++i) {
55         for (size_t j = 0; j < bodies_.size(); ++j) {
56             if (i != j) {
57                 double dx = bodies_[j]->getPosition().x
58                     - bodies_[i]->getPosition().x;
59                 double dy = bodies_[j]->getPosition().y
60                     - bodies_[i]->getPosition().y;
61                 double distSquared = dx * dx + dy * dy;
62                 double dist = std::sqrt(distSquared);
63
64                 if (dist > 0) {
65                     double force = (G * bodies_[i]->getMass()
66                         * bodies_[j]->getMass()) / distSquared;
67                     netForceX[i] += force * (dx / dist);
68                     netForceY[i] += force * (dy / dist);
69                 }
70             }
71         }
72     }
73
74     // Update velocities and positions
75     for (size_t i = 0; i < bodies_.size(); ++i) {
76         double ax = netForceX[i] / bodies_[i]->getMass();
77         double ay = netForceY[i] / bodies_[i]->getMass();
78
79         sf::Vector2f newVelocity = bodies_[i]->getVelocity()
80             + sf::Vector2f(ax * dt, ay * dt);
81         sf::Vector2f newPosition = bodies_[i]->getPosition()
82             + newVelocity * static_cast<float>(dt);
83
84         bodies_[i]->setVelocity(newVelocity);
85         bodies_[i]->setPosition(newPosition);
86     }
87 }
88
89 // Update all bodies' sprite positions
90 void Universe::updateBodies() {
91     for (const auto& body : bodies_) {
92         body->syncSpritePosition();
93     }
94 }
95
96 // Draw function
97 void Universe::draw(sf::RenderTarget& target,
98 sf::RenderStates states) const {
99     for (const auto& body : bodies_) {
100         target.draw(*body, states);
101     }
102 }
103

```

```

104 // Operator overloads
105 std::istream& operator>>(std::istream& input, Universe& universe) {
106     int numBodies;
107     double radius;
108
109     input >> numBodies >> radius;
110     universe.setSize(numBodies);
111     universe.setRadius(radius);
112
113     for (int i = 0; i < numBodies; ++i) {
114         auto body = std::make_shared<CelestialBody>();
115         body->setUniverseRadius(radius);
116         input >> *body;
117         universe.addBody(body);
118     }
119
120     return input;
121 }
122
123 std::ostream& operator<<(std::ostream& output, const Universe& universe) {
124     output << universe.getSize() << "\n" << universe.getRadius() << "\n";
125     for (const auto& body : universe.getBodies()) {
126         output << *body;
127     }
128     return output;
129 }
130
131 const CelestialBody& Universe::operator[](size_t index) const {
132     return *bodies_[index];
133 }
134
135 } // namespace NB

```

5 PS4: Sokoban

5.1 Discussion

This project focused on building Part A and Part B of the Sokoban game using C++ and SFML. The idea was to load a Sokoban level from a file, parse it into a 2D grid, and render everything inside an SFML window. I structured the game around a 2D `std::vector<std::string>` to represent the grid, which made it really simple to update positions and map them directly to the screen. For Part A, I got the basic movement working with WASD or arrow keys and made sure the player couldn't walk through walls or fall off the grid. Part B expanded on this by adding crate pushing, undo functionality, a level reset feature, and full win detection logic. I really enjoyed this project because it brought together so many skills: OOP, file I/O, graphics, game logic—and it was incredibly satisfying to see the whole game come together and actually work.

5.2 What I accomplished

In Part A, I got player movement working reliably, with a separate `movePlayer` function that checks wall collisions and updates the grid safely. I overloaded the operator`>>` to make file input easier and made sure the drawing logic kept background and dynamic elements (like the player and crates) separate, so tiles wouldn't accidentally get overwritten. In Part B, I added full crate pushing mechanics with correct collision detection, level resetting with the R key, undoing moves with the Z key, and detecting when the player has won the level. I also implemented a move counter and win message, which made the gameplay experience a lot more polished and fun.

5.3 What I already knew

Before starting, I already knew how to use SFML to create windows and draw shapes or sprites. I also understood how to use vectors, how to separate logic into clean functions, and the basics of event handling with C++. I had some experience working with 2D grids from previous projects, but this was the first time I really had to tie together movement, collision detection, and graphics all at once in a full game.

5.4 What I learned

I learned a lot about how important it is to separate game logic from rendering. Keeping the grid state separate from the display helped prevent bugs like goals getting erased when the player moved. I also learned how SFML handles events and key repeat behavior and how important it is to check for the right event types to avoid things like multiple movements on a single key press. In Part B, I learned how to manage undo history efficiently by storing only the last 20 grid states, and how to carefully update grid tiles so the player and boxes interacted with the environment correctly. I also got a lot more comfortable debugging grid based problems by thinking about the rules of the game, not just what looked wrong visually.

5.5 Challenges

One big challenge early on was getting the player movement to behave correctly. At first, the player would move two spaces instead of one because I was calling `movePlayer` on every event instead of only `KeyPressed` events. Fixing that made everything feel much smoother. Another challenge was in Part B when crates would get "eaten" by the player or fail to move correctly. I realized I needed to restore the background tiles properly and update both the player's new position and the crate's new position at the same time. Getting the win detection working was also a little tricky because I had to check every storage tile against the original level layout, not just count how many crates were on screen. But after careful testing and a few refactors, everything started working as it should.

5.6 Proof of Execution

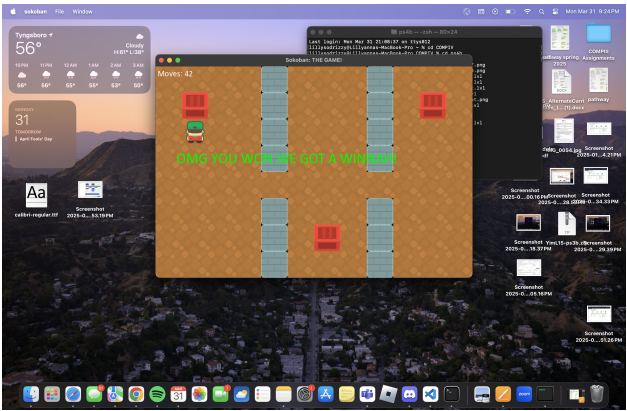


Figure 5: Screenshot of running program

5.7 Code

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -
    lboost_unit_test_framework
4 # Your .hpp files
5 DEPS = Sokoban.hpp
6 # Your compiled .o files
7 OBJECTS = Sokoban.o main.o
8 # The name of your program
9 PROGRAM = Sokoban
10
11 .PHONY: all clean lint Sokoban.a
12
13 all: $(PROGRAM) Sokoban.a
14
15 %.o: %.cpp $(DEPS)
16     $(CC) $(CFLAGS) -c $<
17
18 $(PROGRAM): $(OBJECTS)
19     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
20
21 clean:
22     rm *.o *.a $(PROGRAM)
23
24 lint:
25     cpplint *.cpp *.hpp
26
27 fclean: clean
28     rm -f $(PROGRAM)
29
30 re: fclean all
31
32 Sokoban.a:
33     ar rcs Sokoban.a $(OBJECTS)
```

```
1 // Copyright 2025 Lillyanna Yim
2 #include <iostream>
3 #include <SFML/Audio.hpp>
4 #include <SFML/Graphics.hpp>
5 #include "Sokoban.hpp"
6
7 int main(int argc, char* argv[]) {
```

```

8 // check for correct number of commandline arguments
9 if (argc != 2) {
10     std::cerr << "Usage: ./Sokoban <level_file.lv1>" << std::endl;
11     return 1;
12 }
13
14 // open level file provided by user
15 std::ifstream level(argv[1]);
16 if (!level) {
17     std::cerr << "Error: Level file not found!" << std::endl;
18     return 1;
19 }
20
21 // load level into Sokoban object
22 SB::Sokoban game;
23 try {
24     level >> game; // overloaded operator>>
25     std::cout << "Level loaded successfully!" << std::endl;
26 } catch (const std::exception& e) {
27     std::cerr << "Error loading level: " << e.what() << std::endl;
28     return 1;
29 }
30 level.close();
31
32 // create SFML window based on level size
33 sf::RenderWindow window(
34     sf::VideoMode(game.pixelWidth(), game.pixelHeight()),
35     "Sokoban: THE GAME!");
36
37 window.setKeyRepeatEnabled(false);
38
39 // load the font once
40 sf::Font font;
41 if (!font.loadFromFile("calibri-regular.ttf")) {
42     std::cerr << "Failed to load font\n";
43     return 1;
44 }
45
46 // setup move counter text
47 sf::Text moveText;
48 moveText.setFont(font);
49 moveText.setCharacterSize(20);
50 moveText.setFillColor(sf::Color::White);
51 moveText.setPosition(5.f, 5.f);
52
53 // Load the victory music
54 sf::Music victoryMusic;
55 if (!victoryMusic.openFromFile("2001.wav")) { // path to your uploaded
file
56     std::cerr << "Error loading victory music!" << std::endl;
57     return 1;
58 }
59
60 bool hasWon = false; // To track if the player has won
61
62 while (window.isOpen()) {
63     sf::Event event;
64     while (window.pollEvent(event)) {
65         if (event.type == sf::Event::Closed) {

```

```

66         window.close();
67     }
68
69     if (event.type == sf::Event::KeyPressed) {
70         // Lambda function to handle key presses
71         auto handleKeyPress = [&](sf::Keyboard::Key key) {
72             if (key == sf::Keyboard::W || key == sf::Keyboard::Up) {
73                 game.movePlayer(SB::Direction::Up);
74             }
75             if (key == sf::Keyboard::S || key == sf::Keyboard::Down)
76         {
77             game.movePlayer(SB::Direction::Down);
78         }
79             if (key == sf::Keyboard::A || key == sf::Keyboard::Left)
80         {
81             game.movePlayer(SB::Direction::Left);
82         }
83             if (key == sf::Keyboard::D || key == sf::Keyboard::Right
84         ) {
85                 game.movePlayer(SB::Direction::Right);
86             }
87             if (key == sf::Keyboard::R) {
88                 game.reset();
89                 hasWon = false;
90                 // Stop the music when the game is reset
91                 victoryMusic.stop();
92             }
93             if (key == sf::Keyboard::Z) {
94                 game.undo();
95             }
96         };
97         // Call the lambda to handle the key press
98         handleKeyPress(event.key.code);
99     }
100 }
101
102 // update move counter display
103 moveText.setString("Moves: " + std::to_string(game.getMoveCounter()))
104 );
105
106 // check if the player has won and play the music
107 // Only play the sound and display the message once
108 if (game.hasWon() && !hasWon) {
109     victoryMusic.play();
110     hasWon = true;
111 }
112
113 window.clear();
114 window.draw(game);
115 window.draw(moveText);
116
117 // Display the victory message if the player has won
118 if (hasWon) {
119     sf::Text winText;
120     winText.setFont(font);
121     winText.setCharacterSize(35);
122     winText.setFillColor(sf::Color::Green);
123     winText.setString("OMG YOU WON WE GOT A WINNAH!");
124     winText.setPosition(50.f, 200.f);

```

```

121         window.draw(winText);
122     }
123
124     window.display();
125 }
126
127     return 0;
128 }

```

```

1  // Copyright 2025 Lillyanna Yim
2
3  #pragma once
4  #include <iostream>
5  #include <fstream>
6  #include <vector>
7  #include <string>
8  #include <SFML/Graphics.hpp>
9
10 namespace SB {
11     enum class Direction {
12         Up, Down, Left, Right
13     };
14
15     class Sokoban : public sf::Drawable {
16     public:
17         static const int TILE_SIZE = 64;
18
19         Sokoban();
20         explicit Sokoban(const std::string& filename);
21
22
23         // setters
24         void setHeight(size_t h) { _HEIGHT = h; }
25         void setWidth(size_t w) { _WIDTH = w; }
26         void setGrid(const std::vector<std::string>&g) { _GRID = g; }
27         void setOriginalGrid(const std::vector
28         <std::string>& grid) { _ORIGINALGRID = grid; }
29
30
31         // getters
32         unsigned int pixelHeight() const { return _HEIGHT * TILE_SIZE; }
33         unsigned int pixelWidth() const { return _WIDTH * TILE_SIZE; }
34         unsigned int height() const { return _HEIGHT; }
35         unsigned int width() const { return _WIDTH; }
36         sf::Vector2u playerLoc() const;
37         const std::vector<std::string>& grid() const { return _GRID; }
38         const std::vector<std::string>&
39         originalGrid() const { return _ORIGINALGRID; }
40         bool hasWon() const { return _WINNAH; }
41
42
43         bool isWon() const;
44         void movePlayer(Direction dir);
45         void reset();
46         void undo(); // Optional XC
47         void redo(); // Optional XC
48         int getMoveCounter() const { return _moveCounter; } // XC
49         void incrementMoveCounter() { ++_moveCounter; } // XC
50
51

```

```

52 protected:
53     void draw(sf::RenderTarget& target, sf::RenderStates states) const
        override;
54
55 private:
56     size_t _HEIGHT, _WIDTH;
57     int _moveCounter = 0; // XC
58     std::vector<std::string> _GRID;
59     std::vector<std::string> _ORIGINALGRID;
60     bool _WINNAH = false;
61     std::vector<std::vector<std::string>> _HISTORY; // Optional XC
62     sf::Texture _PLAYER_UP; // Optional XC
63     sf::Texture _PLAYER_DOWN; // Optional XC
64     sf::Texture _PLAYER_LEFT; // Optional XC
65     sf::Texture _PLAYER_RIGHT; // Optional XC
66     Direction _FACING = Direction::Down; // Optional XC
67
68
69
70     // textures
71     sf::Texture _BOX;
72     sf::Texture _EMPTY;
73     sf::Texture _STORAGE;
74     sf::Texture _PLAYER;
75     sf::Texture _WALL;
76 };
77
78 std::ostream& operator<<(std::ostream& out, const Sokoban& s);
79 std::istream& operator>>(std::istream& in, Sokoban& s);
80 } // namespace SB

```

```

1 // Copyright 2025 Lillyanna Yim
2
3 #include <iostream>
4 #include <algorithm>
5 #include <fstream>
6 #include <stdexcept>
7 #include <vector>
8 #include <string>
9 #include "Sokoban.hpp"
10
11 namespace SB {
12
13 // default constructor
14 Sokoban::Sokoban() : _HEIGHT(0), _WIDTH(0) {
15     _WALL.loadFromFile("block_06.png");
16     _EMPTY.loadFromFile("ground_01.png");
17     _BOX.loadFromFile("crate_03.png");
18     _STORAGE.loadFromFile("ground_04.png");
19     _PLAYER_DOWN.loadFromFile("player_05.png");
20     _PLAYER_LEFT.loadFromFile("player_20.png");
21     _PLAYER_RIGHT.loadFromFile("player_17.png");
22     _PLAYER_UP.loadFromFile("player_08.png");
23 }
24
25 // constructor loads level from file
26 Sokoban::Sokoban(const std::string& filename) : _HEIGHT(0), _WIDTH(0) {
27     std::ifstream file(filename);
28     if (!file.is_open()) {
29         throw std::runtime_error("Cannot open file");

```



```

30     }
31     file >> *this; // use operator>> to load level
32
33     _WALL.loadFromFile("block_06.png");
34     _EMPTY.loadFromFile("ground_01.png");
35     _BOX.loadFromFile("crate_03.png");
36     _STORAGE.loadFromFile("ground_04.png");
37
38     _PLAYER_DOWN.loadFromFile("player_05.png");
39     _PLAYER_RIGHT.loadFromFile("player_17.png");
40     _PLAYER_UP.loadFromFile("player_08.png");
41     _PLAYER_LEFT.loadFromFile("player_20.png");
42 }
43
44 // find current location of the player '@' symbol
45 sf::Vector2u Sokoban::playerLoc() const {
46     for (size_t y = 0; y < _HEIGHT; ++y) {
47         for (size_t x = 0; x < _WIDTH; ++x) {
48             if (_GRID[y][x] == '@') {
49                 return {static_cast<unsigned int>(x), static_cast<unsigned
50 int>(y)};
51             }
52         }
53         throw std::runtime_error("Player not found");
54     }
55
56 // draw function using SFML
57 void Sokoban::draw(sf::RenderTarget& target, sf::RenderStates states) const
58 {
59     for (size_t y = 0; y < _HEIGHT; ++y) {
60         for (size_t x = 0; x < _WIDTH; ++x) {
61             sf::Sprite tile1, tile2;
62             char c = _GRID[y][x];
63
64             tile1.setTexture(_EMPTY); // base layer
65             bool draw2 = false;      // second layer (box or player)
66
67             switch (c) {
68                 case '@':
69                     switch (_FACING) {
70                         case Direction::Up:
71                             tile2.setTexture(_PLAYER_UP);
72                             break;
73                         case Direction::Down:
74                             tile2.setTexture(_PLAYER_DOWN);
75                             break;
76                         case Direction::Left:
77                             tile2.setTexture(_PLAYER_LEFT);
78                             break;
79                         case Direction::Right:
80                             tile2.setTexture(_PLAYER_RIGHT);
81                             break;
82                     }
83                     draw2 = true;
84                     break;
85                 case '.':
86                     break;

```

```

87
88         case '#':
89             tile1.setTexture(_WALL);
90             break;
91
92         case 'A':
93             tile2.setTexture(_BOX);
94             draw2 = true;
95             break;
96
97         case 'a':
98             tile1.setTexture(_STORAGE);
99             break;
100
101         default:
102             break;
103     }
104
105     tile1.setPosition(x * TILE_SIZE, y * TILE_SIZE);
106     target.draw(tile1, states);
107
108     if (draw2) {
109         tile2.setPosition(x * TILE_SIZE, y * TILE_SIZE);
110         target.draw(tile2, states);
111     }
112 }
113 }
114 }
115
116 // output
117 std::istream& operator>>(std::istream& in, Sokoban& s) {
118     size_t h, w;
119     if (!(in >> h >> w)) {
120         throw std::runtime_error("Invalid grid dimensions");
121     }
122
123     s.setHeight(h);
124     s.setWidth(w);
125
126     std::vector<std::string> grid;
127     std::string line;
128     std::getline(in, line);
129
130     for (size_t i = 0; i < h; ++i) {
131         if (!std::getline(in, line)) {
132             throw std::runtime_error("Invalid grid data");
133         }
134         grid.push_back(line);
135     }
136     s.setGrid(grid);
137     s.setOriginalGrid(grid);
138     return in;
139 }
140
141 std::ostream& operator<<(std::ostream& out, const Sokoban& s) {
142     out << s.height() << " " << s.width() << "\n";
143     for (const auto& row : s.grid()) {
144         out << row << "\n";
145     }

```

```

146     return out;
147 }
148
149 void SB::Sokoban::movePlayer(Direction dir) {
150     if (_WINNAH) return;
151     _FACING = dir;
152     _HISTORY.push_back(_GRID);
153
154     if (_HISTORY.size() > 20) {
155         _HISTORY.erase(_HISTORY.begin());
156     }
157
158     sf::Vector2u player = playerLoc();
159     int px = player.x;
160     int py = player.y;
161
162     int dx = 0, dy = 0;
163     switch (dir) {
164         case Direction::Up:    dy = -1; break;
165         case Direction::Down:  dy = 1;  break;
166         case Direction::Left:  dx = -1; break;
167         case Direction::Right: dx = 1;  break;
168     }
169
170     int newX = px + dx;
171     int newY = py + dy;
172
173     if (newX < 0 || newY < 0 || newX >= static_cast<int>(_WIDTH)
174         || newY >= static_cast<int>(_HEIGHT))
175         return;
176
177     char& dest = _GRID[newY][newX];
178     char& curr = _GRID[py][px];
179
180     if (dest == '#') return;
181
182     if (dest == 'A') {
183         int boxX = newX + dx;
184         int boxY = newY + dy;
185
186         if (boxX < 0 || boxY < 0 || boxX >= static_cast<int>(_WIDTH) ||
187             boxY >= static_cast<int>(_HEIGHT))
188             return;
189
190         char& box = _GRID[boxY][boxX];
191
192         if (box == '#' || box == 'A') return;
193
194         _GRID[boxY][boxX] = (_ORIGINALGRID[boxY][boxX] == 'a') ? 'A' : 'A';
195
196         // Restore the tile the box was on (goal or floor)
197         _GRID[newY][newX] = (_ORIGINALGRID[newY][newX] == 'a') ? 'a' : '.';
198     }
199
200     curr = (_ORIGINALGRID[py][px] == 'a') ? 'a' : '.';
201     _GRID[newY][newX] = '@';
202
203     for (const auto& row : _GRID) {
204         std::cout << row << std::endl;

```

```

205     }
206     std::cout << "----" << std::endl;
207
208     if (isWon()) {
209         _WINNAH = true;
210     }
211
212     incrementMoveCounter();
213 }
214
215 bool SB::Sokoban::isWon() const {
216     for (size_t y = 0; y < _HEIGHT; ++y) {
217         for (size_t x = 0; x < _WIDTH; ++x) {
218             if (_ORIGINALGRID[y][x] == 'a') {
219                 if (_GRID[y][x] != 'A') {
220                     std::cout << "GOAL at (" << x << ", " << y << ") is
missing a box, found: " << _GRID[y][x] << std::endl;
221                     return false;
222                 }
223             }
224         }
225     }
226     std::cout << "WIN DETECTED!" << std::endl;
227     return true;
228 }
229
230 void SB::Sokoban::reset() {
231     _GRID = _ORIGINALGRID;
232     _moveCounter = 0;
233     _WINNAH = false;
234 }
235
236 void SB::Sokoban::undo() {
237     if (_HISTORY.empty()) return;
238     _GRID = _HISTORY.back();
239     _HISTORY.pop_back();
240     _moveCounter = std::max(0, _moveCounter - 1);
241 }
242
243 } // namespace SB

```

6 PS5: DNA String Alignment

6.1 Discussion

This project was all about building a program to compute the edit distance between two DNA sequences using dynamic programming, specifically the Needleman-Wunsch algorithm. The goal was to figure out the minimum number of edits needed to align the sequences and get a sense of how genetically similar they are. We had to fill a 2D vector as a matrix and trace back the alignment path based on penalties for matches, mismatches, and gaps. On top of getting the program working, we also had to analyze how much time and memory it used. This project pushed us pretty hard because it involved a lot more algorithmic thinking and data analysis than anything we'd done before.

6.2 What I accomplished

We successfully implemented the Needleman-Wunsch algorithm using a 2D vector array to store the dynamic programming matrix. We built a working alignment function that takes two DNA strings, fills out the matrix according to match/mismatch/gap penalties, and traces the path back to generate the alignment. We also wrote unit tests to check if our distance calculations and output matched the expected results. After building the program, we ran performance tests, measured time and memory usage, and even hand-calculated a few edge cases to confirm the program worked correctly. It was a huge learning experience but also really rewarding to see everything finally line up.

6.3 What I already knew

Before this project, we already had a decent grasp of how dynamic programming works conceptually. We also knew how to set up 2D vectors in C++, how to use if statements to manage different conditions, and how to loop through matrices. But we hadn't really applied these skills to something as complex and math-heavy as DNA sequence alignment before. I had never heard of edit distance algorithms before, but I was curious to know exactly how they worked within this project.

6.4 What I learned

We learned how the Needleman-Wunsch algorithm actually works in practice and how to translate a step-by-step paper explanation into working code. We also learned how to handle tricky edge cases where the sequences have lots of gaps or mismatches. On top of that, we got a lot better at thinking about memory and time efficiency, especially since some of the DNA files we tested were huge. Watching tutorials and manually working through small examples by hand helped solidify our understanding. Another important thing we learned was how valuable it is to debug carefully and use simple test cases before trying large inputs.

6.5 Challenges

The hardest part of the project was not writing the code. It was understanding the problem itself. At first, the algorithm seemed super confusing and we struggled to figure out exactly how to set up the matrix and backtrack correctly. Once we understood the rules and penalties clearly, turning that into code wasn't too bad. Another challenge was figuring out what tests to write. Honestly, at first we just guessed what would probably be tested, submitted to the autograder, and used the feedback to adjust our own tests. It felt a little messy at first, but it helped us cover the important cases in the end.

6.6 Code

```
1 CC = g++
2 CFLAGS = -std=c++11 -Wall -Wextra -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system
4 BOOST_LIB = -lboost_unit_test_framework
5 # Your .hpp files
```

```

6 DEPS = EDistance.hpp
7 # Your compiled .o files
8 OBJECTS = EDistance.o
9 # The name of your program
10 PROGRAM = EDistance
11
12 .PHONY: all clean lint
13
14 all: $(PROGRAM) EDistance.a test
15
16 %.o: %.cpp $(DEPS)
17     $(CC) $(CFLAGS) -c $<
18
19 $(PROGRAM): main.o $(OBJECTS)
20     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
21
22 test: test.o $(OBJECTS)
23     $(CC) $(CFLAGS) -o $@ $^ $(LIB) $(BOOST_LIB)
24
25 EDistance.a: $(OBJECTS)
26     ar rcs $@ $^
27
28 clean:
29     rm -f *.o $(PROGRAM) EDistance.a test
30
31 lint:
32     cpplint *.cpp *.hpp

```

```

1 // Copyright 2025 Vanzel Essien, Lillyanna Yim
2
3 #include <iostream>
4 #include <iomanip>
5 #include <string>
6 #include "EDistance.hpp"
7 #include <SFML/System.hpp>
8
9 int main() {
10     sf::Clock clock; // start the clock
11
12     std::string s1, s2; // two genetic sequences
13     std::cin >> s1 >> s2; // load the sequences from input
14     EDistance ed(s1, s2); // Edit Distance object
15     int distance = ed.optDistance(); // get edit distance
16     std::string alignment = ed.alignment(); // get the alignment
17
18     // output the results
19     std::cout << "Edit Distance = " << distance << std::endl;
20     std::cout << alignment << std::endl; // ed.outputMatrix();
21
22     // stop the clock and output the elapsed time
23     sf::Time elapsed = clock.getElapsedTime();
24     std::cout << "Execution time is " << elapsed.asSeconds() << " seconds"
25     << std::endl;
26     return 0;
27 }

```

```

1 // Copyright 2025 Vanzel Essien, Lillyanna Yim
2 #pragma once
3 #include <iostream>
4 #include <iomanip>

```

```

5  #include <algorithm>
6  #include <string>
7  #include <vector>
8
9  class EDistance {
10 public:
11     EDistance(const std::string& s1, const std::string& s2);
12
13     static int penalty(char a, char b);
14     static int min3(int a, int b, int c);
15
16     int optDistance();
17     std::string alignment();
18
19     // getters
20     std::vector<std::vector<int>> getMatrix() const { return matrix; }
21     // setters
22
23     // friends
24     friend std::istream& operator>>(std::ostream& is, EDistance& ed);
25     friend std::ostream& operator<<(std::ostream& os, const EDistance& ed);
26
27     // others
28     void outputMatrix() const; // configure output
29
30 private:
31     std::string s1;
32     std::string s2;
33     int match = 0;
34     int mismatch = 1;
35     int gap = 2;
36     std::vector<std::vector<int>> matrix;
37 };

```

```

1  // Copyright 2025 Vanzel Essien, Lillyanna Yim
2
3  #include "EDistance.hpp"
4  #include <iostream>
5  #include <string>
6  #include <vector>
7  #include <algorithm>
8
9  // Constructor
10 EDistance::EDistance(const std::string& s1, const std::string& s2)
11     : s1(s1), s2(s2) {
12     // init matrix using string sizes
13     matrix.resize(s1.length() + 1, std::vector<int>(s2.length() + 1, 0));
14     // begin filling matrix with gap penalties
15     for (size_t i = 0; i <= s1.length(); ++i) {
16         matrix[i][0] = i * gap;
17     }
18     for (size_t j = 0; j <= s2.length(); ++j) {
19         matrix[0][j] = j * gap;
20     }
21 }
22
23 // Returns the penalty for a mismatch or match
24 int EDistance::penalty(char a, char b) {
25     if (a == b) {
26         return 0; // match

```

```

27     } else if (a == '-' || b == '-') {
28         return 2; // gap
29     } else {
30         return 1; // mismatch
31     }
32 }
33
34 // Returns the minimum of three integers
35 int EDistance::min3(int a, int b, int c) {
36     return std::min({a, b, c}); // min algorithm
37 }
38 // Returns the edit distance between two strings
39 int EDistance::optDistance() {
40     for (size_t i = 1; i <= s1.length(); ++i) {
41         for (size_t j = 1; j <= s2.length(); ++j) {
42             int cost = penalty(s1[i - 1], s2[j - 1]);
43             matrix[i][j] = min3(matrix[i - 1][j] + gap,
44                                 matrix[i][j - 1] + gap,
45                                 matrix[i - 1][j - 1] + cost);
46         }
47     }
48     return matrix[s1.length()][s2.length()];
49 }
50 // traces back through the matrix to find the optimal alignment
51 std::string EDistance::alignment() {
52     std::string aligned_s1, aligned_s2;
53     int i = s1.length();
54     int j = s2.length();
55
56     while (i > 0 || j > 0) { // while there are characters to align
57         if (i > 0 && j > 0 && matrix[i][j] == matrix[i - 1][j - 1] + penalty
58             (s1[i - 1]
59              , s2[j - 1])) {
60             aligned_s1 += s1[i - 1];
61             aligned_s2 += s2[j - 1];
62             --i;
63             --j;
64         } else if (i > 0 && matrix[i][j] == matrix[i - 1][j] + gap) {
65             aligned_s1 += s1[i - 1];
66             aligned_s2 += '-';
67             --i;
68         } else {
69             aligned_s1 += '-';
70             aligned_s2 += s2[j - 1];
71             --j;
72         }
73     }
74
75     // Reverse the aligned strings to get the correct order (built backwards)
76     std::reverse(aligned_s1.begin(), aligned_s1.end());
77     std::reverse(aligned_s2.begin(), aligned_s2.end());
78
79     // format the output
80     std::string output = "";
81     for (size_t i = 0; i < aligned_s1.length() && i < aligned_s2.length();
82         ++i) {
83         output += aligned_s1[i];
84         output += " ";
85     }
86 }

```



```

83     output += aligned_s2[i];
84     output += " ";
85     output += std::to_string(penalty(aligned_s1[i], aligned_s2[i]));
86     output += "\n";
87 }
88 return output;
89 }
90 void EDistance::outputMatrix() const {
91     for (const auto& row : matrix) {
92         for (const auto& val : row) {
93             std::cout << std::setw(3) << val;
94         }
95         std::cout << std::endl;
96     }
97 }

```

7 PS6: RandWriter

7.1 Discussion

This project was about building a k-order Markov model to analyze text and generate new random text that mimics the original patterns. I created a RandWriter class that takes a training string and constructs a frequency map of all kgrams and the characters that follow them. Using this model, the program can generate text that statistically follows the style of the original input. It was interesting to see how simple statistical modeling could create surprisingly realistic outputs, especially when trained on structured sources like books or lyrics. Working through this project gave me a much deeper appreciation for how even basic probability models can capture the "feel" of natural language.

7.2 What I accomplished

I implemented a complete RandWriter class that builds the Markov model at the character level. It handles frequency tracking, random character selection, and text generation based on the training string. I made sure to follow all the assignment guidelines, including using `std::mt19937` with a fixed seed of 42 for deterministic results. I also wrote a clean overload for the operator« and structured the code to be modular; with careful exception handling for edge cases. On top of that, I completed the extra credit by developing a separate word-level model called WordWriter, which applies the same concepts but works at the level of whole words rather than individual characters.

7.3 What I already knew

Before starting this project, I was already familiar with basic random number generation in C++. Working with maps and vectors, and designing classes with clean interfaces. I also had some background in general probability and knew a little bit about how Markov chains work conceptually. However, I hadn't built a full Markov model from scratch before and I hadn't dealt with making randomness fully deterministic for testing purposes.

7.4 What I learned

I learned how to use `std::mt19937` properly and why it's important to fix the seed when reproducible results are needed. I also learned how to correctly implement wraparound behavior for kgrams, which was trickier than I expected at first. On the design side, I learned how to make my code more modular and maintainable by using helper functions and lambda expressions where appropriate. Building the extra credit version at the word level also helped me practice thinking about abstraction—how the same core ideas can be applied whether you're working with characters or full words.

7.5 Challenges

One of the first issues I ran into was mistakenly using `std::randomdevice` instead of `std::mt19937`, which gave inconsistent output and wasn't allowed by the project rules. Once I caught that mistake, switching to `mt19937` fixed the problem. Another big challenge was handling the circular nature of the kgrams. At first, my model missed some of the wraparound kgrams because I wasn't extending the input string properly. After rethinking the logic and appending the first k characters to the end of the string, I was able to fix the issue. Debugging these mistakes helped me understand the importance of small details when building statistical models.

7.6 Code

```
1 CC = g++
2 CFLAGS = --std=c++17 -Wall -Werror -pedantic -g
3 LIB = -lsfml-graphics -lsfml-audio -lsfml-window -lsfml-system -
      lboost_unit_test_framework
4 # Your .hpp files
```

```

5  DEPS = RandWriter.hpp
6  # Your compiled .o files
7  OBJECTS = RandWriter.o TextWriter.o
8  # The name of your program
9  PROGRAM = TextWriter
10
11 .PHONY: all clean lint TextWriter.a test
12
13 all: $(PROGRAM) test TextWriter.a
14
15 %.o: %.cpp $(DEPS)
16     $(CC) $(CFLAGS) -c $<
17
18 $(PROGRAM): $(OBJECTS)
19     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
20
21 clean:
22     rm *.o *.a $(PROGRAM) test
23
24 lint:
25     cpplint *.cpp *.hpp
26
27 fclean: clean
28     rm -f $(PROGRAM) test
29
30 re: fclean all
31
32 test: test.o RandWriter.o
33     $(CC) $(CFLAGS) -o $@ $^ $(LIB)
34
35 TextWriter.a:
36     ar rcs TextWriter.a $(OBJECTS) test.o

```

```

1  // Copyright 2025 Lillyanna Yim
2
3  #ifndef RANDWRITER_HPP
4  #define RANDWRITER_HPP
5
6  #include <string>
7  #include <map>
8  #include <random>
9  #include <iostream>
10
11 class RandWriter {
12 public:
13     RandWriter(const std::string& text, size_t k);
14     size_t orderK() const;
15     int freq(const std::string& kgram) const;
16     int freq(const std::string& kgram, char c) const;
17     char kRand(const std::string& kgram);
18     std::string generate(const std::string& kgram, size_t L);
19
20     friend std::ostream& operator<<(std::ostream& out, const RandWriter& rw)
21     ;
22 private:
23     std::string text_;
24     size_t k_;
25     std::string alphabet_;
26     std::map<std::string, std::map<char, int>>> model_;

```

```

27     std::mt19937 gen_;
28 };
29
30 #endif // RANDWRITER_HPP

```

```

1  // Copyright 2025 Lillyanna Yim'
2
3  #include "RandWriter.hpp"
4  #include <stdexcept>
5  #include <numeric>
6  #include <string>
7  #include <vector>
8
9
10 RandWriter::RandWriter(const std::string& text, size_t k)
11 : text_(text), k_(k), gen_(42) {
12     if (text.size() < k) throw
13         std::invalid_argument("Text length must be >= k");
14
15     // create a circular text
16     std::string circularText = text + text.substr(0, k);
17
18     // iterate through the text to build the model
19     for (size_t i = 0; i < text.size(); ++i) {
20         std::string kgram = circularText.substr(i, k);
21         char nextChar = circularText[i + k];
22         // increment the frequency of nextChar after kgram
23         model_[kgram][nextChar]++;
24
25         // track unique characters in the alphabet
26         if (alphabet_.find(nextChar) == std::string::npos) {
27             alphabet_ += nextChar;
28         }
29     }
30 }
31
32 size_t RandWriter::orderK() const {
33     return k_;
34 }
35
36 // returns the frequency of the kgram in the model
37 int RandWriter::freq(const std::string& kgram) const {
38     if (kgram.length() != k_) throw
39         std::invalid_argument("Invalid kgram length");
40     auto it = model_.find(kgram);
41     if (it == model_.end()) return 0;
42
43     int sum = 0;
44     for (const auto& p : it->second) sum += p.second;
45     return sum;
46 }
47
48 // if kgram is not found, returns 0
49 int RandWriter::freq(const std::string& kgram, char c) const {
50     if (kgram.length() != k_) throw
51         std::invalid_argument("Invalid kgram length");
52     auto it = model_.find(kgram);
53     if (it == model_.end()) return 0;
54
55     auto chIt = it->second.find(c);

```

```

56     return (chIt == it->second.end()) ? 0 : chIt->second;
57 }
58
59 // if kgram is found, returns the frequency of c after kgram
60 char RandWriter::kRand(const std::string& kgram) {
61     if (kgram.length() != k_) throw
62     std::invalid_argument("Invalid kgram length");
63
64     auto it = model_.find(kgram);
65     if (it == model_.end()) throw std::invalid_argument("kgram not found");
66
67     const auto& charFreqs = it->second;
68     std::vector<char> choices;
69
70     for (const auto& [ch, count] : charFreqs) {
71         choices.insert(choices.end(), count, ch);
72     }
73
74     if (choices.empty()) throw
75     std::runtime_error("No characters follow the given kgram");
76
77     std::uniform_int_distribution<> dist(0, choices.size() - 1);
78     return choices[dist(gen_)];
79 }
80
81 // generates a random string of length L starting with kgram
82 std::string RandWriter::generate(const std::string& kgram, size_t L) {
83     if (kgram.length() != k_) throw
84     std::invalid_argument("Invalid kgram length");
85     if (L < k_) throw std::invalid_argument("Length must be at least k");
86
87     std::string result = kgram;
88     std::string current = kgram;
89
90     while (result.length() < L) {
91         char next = kRand(current);
92         result += next;
93         current = current.substr(1) + next;
94     }
95     return result;
96 }
97
98 // overloads the << operator to print the RandWriter object
99 std::ostream& operator<<(std::ostream& out, const RandWriter& rw) {
100     out << "Order: " << rw.k_ << '\n';
101     out << "Alphabet: " << rw.alphabet_ << '\n';
102     out << "Frequencies:\n";
103
104     for (const auto& [kgram, nextMap] : rw.model_) {
105         for (const auto& [ch, count] : nextMap) {
106             out << kgram << ", " << ch << ": " << count << '\n';
107         }
108     }
109
110     return out;
111 }

```

```

1 // Copyright 2025 Lillyanna Yim
2
3 #include <iostream>

```

```

4 #include <string>
5 #include "RandWriter.hpp"
6
7 int main(int argc, char* argv[]) {
8     // expecting k and L
9     if (argc != 3) {
10         std::cerr << "Usage: " << argv[0] << " <k> <L>\n";
11         return 1;
12     }
13
14     // parse command-line arguments: k (order) and L (length of output)
15     size_t k = std::stoi(argv[1]);
16     size_t L = std::stoi(argv[2]);
17
18     // read the entire input text from standard input
19     std::string inputText((std::istreambuf_iterator<char>(std::cin)),
20     std::istreambuf_iterator<char>());
21
22     // check if input text is empty
23     if (inputText.length() < k) {
24         std::cerr << "Input text must be at least length k.\n";
25         return 1;
26     }
27
28     try {
29         // create a RandWriter object with the input text and k
30         RandWriter rw(inputText, k);
31         // generate a random string of length L using the first k characters
32         std::string seed = inputText.substr(0, k);
33         // generate the output string
34         std::string output = rw.generate(seed, L);
35         // print the generated string
36         std::cout << output << std::endl;
37     } catch (const std::exception& e) { // catch any exceptions
38         std::cerr << "Error: " << e.what() << std::endl;
39         return 1;
40     }
41
42     return 0;
43 }

```

```

1 // Copyright 2025 Lillyanna Yim
2
3 #include <iostream>
4 #include <vector>
5 #include <string>
6 #include "WordWriter.hpp"
7
8 int main() {
9     std::string inputText =
10         "are you ready kids aye aye captain i cant hear you aye aye captain
11         "
12         "oh who lives in a pineapple under the sea spongebob squarepants "
13         "absorbent and yellow and porous is he spongebob squarepants";
14
15     size_t k = 2;
16     size_t L = 15;
17
18     WordWriter ww(inputText, k);

```

```

19     std::vector<std::string> seed = {"aye", "aye"};
20     std::string output = ww.generate(seed, L);
21
22     std::cout << "Generated SpongeBob text:\n" << output << std::endl;
23
24     return 0;
25 }

```

```

1  // Copyright 2025 Lillyanna Yim
2
3  #ifndef WORDWRITER_HPP
4  #define WORDWRITER_HPP
5
6  #include <string>
7  #include <vector>
8  #include <map>
9  #include <random>
10
11 class WordWriter {
12 public:
13     WordWriter(const std::string& text, size_t k);
14     size_t orderK() const;
15     int freq(const std::vector<std::string>& kgram) const;
16     int freq(const std::vector<std::string>&
17             kgram, const std::string& word) const;
18     std::string kRand(const std::vector<std::string>& kgram);
19     std::string generate(const std::vector<std::string>& kgram, size_t L);
20
21 private:
22     size_t k_;
23     std::vector<std::string> words_;
24     std::map<std::vector<std::string>, std::map<std::string, int>> model_;
25     std::mt19937 gen_;
26 };
27
28 #endif // WORDWRITER_HPP

```

```

1  // Copyright 2025 Lillyanna Yim
2
3  #include "WordWriter.hpp"
4  #include <iostream>
5  #include <sstream>
6  #include <string>
7  #include <vector>
8  #include <map>
9  #include <random>
10 #include <stdexcept>
11
12 class WordWriter {
13 public:
14     // constructor builds model from input text and order k
15     WordWriter(const std::string& text, size_t k);
16
17     // returns the order of model
18     size_t orderK() const;
19
20     // returns frequency of given kgram
21     int freq(const std::vector<std::string>& kgram) const;
22
23     // returns how often a specific word follows given kgram

```

```

24     int freq(const std::vector<std::string>& kgram,
25             const std::string& word) const;
26
27     // randomly selects word that follows kgram; weighted by frequency
28     std::string kRand(const std::vector<std::string>& kgram);
29
30     // regenerates text of length L using model starting from kgram
31     std::string generate(const std::vector<std::string>& kgram, size_t L);
32
33 private:
34     size_t k_;    // order of model
35     std::vector<std::string> words_; // all words from input text
36     std::map<std::vector<std::string>,
37             std::map<std::string, int>>> model_; // Markov model
38     std::mt19937 gen_; // random number generator
39 };
40
41 // constructor builds k-order word-based model with wraparound
42 WordWriter::WordWriter(const std::string& text, size_t k) : k_(k), gen_(42)
43 {
44     std::istringstream iss(text);
45     std::string word;
46
47     // split input text into words
48     while (iss >> word) words_.push_back(word);
49
50     if (words_.size() < k)
51         throw std::invalid_argument("Not enough words for k-gram");
52
53     // build the model with wraparound logic
54     for (size_t i = 0; i < words_.size(); ++i) {
55         std::vector<std::string> kgram;
56
57         // build the kgram starting at index i (wrap around using %)
58         for (size_t j = 0; j < k; ++j) {
59             kgram.push_back(words_[(i + j) % words_.size()]);
60         }
61
62         // get the next word after the kgram (also wrapped around)
63         std::string nextWord = words_[(i + k) % words_.size()];
64
65         // increment frequency of nextWord after this kgram
66         model_[kgram][nextWord]++;
67     }
68
69 // returns the order k of the model
70 size_t WordWriter::orderK() const {
71     return k_;
72 }
73
74 // returns total frequency of a kgram
75 int WordWriter::freq(const std::vector<std::string>& kgram) const {
76     if (kgram.size() != k_)
77         throw std::invalid_argument("Invalid kgram size");
78
79     auto it = model_.find(kgram);
80     if (it == model_.end()) return 0;
81

```



```

82     int sum = 0;
83     for (const auto& p : it->second) sum += p.second;
84     return sum;
85 }
86
87 // returns how often a specific word follows the given kgram
88 int WordWriter::freq(const std::vector<std::string>& kgram,
89                     const std::string& word) const {
90     if (kgram.size() != k_)
91         throw std::invalid_argument("Invalid kgram size");
92
93     auto it = model_.find(kgram);
94     if (it == model_.end()) return 0;
95
96     auto wit = it->second.find(word);
97     return (wit == it->second.end()) ? 0 : wit->second;
98 }
99
100 // randomly chooses a word that follows the kgram, weighted by frequency
101 std::string WordWriter::kRand(const std::vector<std::string>& kgram) {
102     if (kgram.size() != k_)
103         throw std::invalid_argument("Invalid kgram size");
104
105     auto it = model_.find(kgram);
106     if (it == model_.end())
107         throw std::invalid_argument("kgram not found");
108
109     const auto& nextWords = it->second;
110     std::vector<std::string> pool;
111
112     // flatten the frequency map into a vector for weighted random selection
113     for (const auto& [word, count] : nextWords) {
114         pool.insert(pool.end(), count, word);
115     }
116
117     if (pool.empty())
118         throw std::runtime_error("No words follow the kgram");
119
120     std::uniform_int_distribution<> dist(0, pool.size() - 1);
121     return pool[dist(gen_)];
122 }
123
124 // generates L words of text based on the Markov model
125 std::string WordWriter::generate(const
126 std::vector<std::string>& kgram, size_t L) {
127     if (kgram.size() != k_)
128         throw std::invalid_argument("Invalid kgram size");
129
130     if (L < k_)
131         throw std::invalid_argument("L must be >= k");
132
133     std::vector<std::string> result = kgram;
134     std::vector<std::string> current = kgram;
135
136     // generate words until the desired length is reached
137     while (result.size() < L) {
138         std::string next = kRand(current);
139         result.push_back(next);
140         current.erase(current.begin()); // slide the window

```

```

141     current.push_back(next); // add the new word
142 }
143
144 // combine result vector into a single space separated string
145 std::ostringstream oss;
146 for (const std::string& w : result) oss << w << ' ';
147 return oss.str();
148 }

```

```

1 // Copyright 2025 Lillyanna Yim
2
3 #include <iostream>
4 #include <string>
5 #include <sstream>
6 #include "RandWriter.hpp"
7
8 #define BOOST_TEST_DYN_LINK
9 #define BOOST_TEST_MODULE RandomWriterTests
10 #include <boost/test/unit_test.hpp>
11
12 // test that the constructor stores the order correctly
13 // check orderk() returns the expected value
14 BOOST_AUTO_TEST_CASE(testConstructorAndOrderK) {
15     RandWriter model("hellohello", 2);
16     BOOST_REQUIRE_EQUAL(model.orderK(), 2);
17 }
18
19 // test frequency of valid and invalid kgrams
20 // ensure exception is thrown for incorrect length
21 BOOST_AUTO_TEST_CASE(testFreqSingleParam) {
22     RandWriter model("ababab", 2);
23     BOOST_REQUIRE_EQUAL(model.freq("ab"), 3);
24     BOOST_REQUIRE_EQUAL(model.freq("zz"), 0);
25     BOOST_REQUIRE_THROW(model.freq("a"), std::invalid_argument);
26 }
27
28 // test frequency of specific character following a kgram
29 // check exception for kgram with wrong length
30 BOOST_AUTO_TEST_CASE(testFreqTwoParams) {
31     RandWriter model("abcabc", 1);
32     BOOST_REQUIRE_EQUAL(model.freq("a", 'b'), 2);
33     BOOST_REQUIRE_EQUAL(model.freq("b", 'c'), 2);
34     BOOST_REQUIRE_EQUAL(model.freq("x", 'y'), 0);
35     BOOST_REQUIRE_THROW(model.freq("ab", 'c'), std::invalid_argument);
36 }
37
38 // verify krand returns valid following characters
39 // confirm exceptions for invalid or unknown kgrams
40 BOOST_AUTO_TEST_CASE(testKRandOutputValid) {
41     RandWriter model("xyzxyz", 1);
42     char next = model.kRand("x");
43     BOOST_REQUIRE((next == 'y' || next == 'x' || next == 'z'));
44     BOOST_REQUIRE_THROW(model.kRand("xy"), std::invalid_argument);
45     BOOST_REQUIRE_NO_THROW(model.kRand("y"));
46 }
47
48 // test that generate creates correct output length
49 // check for correct starting kgram and valid inputs
50 BOOST_AUTO_TEST_CASE(testGenerateBasic) {
51     RandWriter model("abcabc", 1);

```

```

52     std::string result = model.generate("a", 4);
53     BOOST_REQUIRE_EQUAL(result.length(), 4);
54     BOOST_REQUIRE_EQUAL(result.substr(0, 1), "a");
55     BOOST_REQUIRE_THROW(model.generate("ab", 4), std::invalid_argument);
56     BOOST_REQUIRE_NO_THROW(model.generate("b", 4));
57 }
58
59 // validate order-0 model handles empty string
60 // confirm freq returns total & per character counts
61 BOOST_AUTO_TEST_CASE(testOrderZeroModel) {
62     RandWriter model("aabbbaac", 0);
63     BOOST_REQUIRE_EQUAL(model.freq(""), 7);
64     BOOST_REQUIRE_EQUAL(model.freq("", 'a'), 4);
65     BOOST_REQUIRE_EQUAL(model.freq("", 'c'), 1);
66     BOOST_REQUIRE_THROW(model.freq("a"), std::invalid_argument);
67 }

```

8 PS7: Kronos Log Parsing

8.1 Discussion

This project involved parsing and analyzing log files from the Kronos InTouch time clock system. Each log file contains timestamps for when the device boots up or restarts and the goal was to identify and match each boot start with a corresponding completion event. Once a valid boot sequence was detected, the program calculated the time taken to complete that boot in milliseconds and saved the results to a formatted .rpt file. The final report includes a summary header and a list of all completed and incomplete boots. The script was written in Python, which made it easier to handle file parsing and pattern matching compared to using C++. It was interesting to work with real-world logs and produce structured output from raw data.

8.2 What I accomplished

I wrote a complete Python script that processes each .log file and generates a corresponding .rpt file summarizing boot events. Using regular expressions, I identified boot start and completion lines then extracted their timestamps and calculated the duration between them in milliseconds. I structured the code to handle edge cases like overlapping boots or missing completion entries. I also implemented a summary section at the top of each report that shows how many boots were attempted and how many finished successfully. The final output matched the format provided in the assignment and handled both small and large log files efficiently.

8.3 What I already knew

Before starting the project, I already had some experience using Python. This including reading files, working with regular expressions, and using the datetime module. I was also familiar with writing scripts that process line-by-line data and had a general idea of how boot logs work based on past coursework. However, I had never written a script that parsed real system logs or generated formal report files before so this project was a good step up from previous work.

8.4 What I learned

I learned how to use regular expressions to reliably extract patterns from large files and how to match related events across many lines of text. I also learned how to manage state in a script keeping track of which boot starts had not yet been completed and carefully handling situations where multiple boots were listed in a row. On the reporting side, I learned how to format clean and readable output in a custom format and how to keep track of summary statistics like total and completed boots. This was also one of my first times working with log parsing in a real-world context and it gave me a better understanding of how devices log activity.

8.5 Challenges

The hardest part was figuring out how to correctly pair boot start lines with their corresponding completion lines, especially when the logs had interruptions or overlapping sequences. At first, I struggled with how to structure the data so I could keep track of multiple active or incomplete boots. I also had to think carefully about how to handle large files without using too much memory or getting bogged down. I learned to break the problem into smaller pieces first by identifying the boot markers, then parsing timestamps, then calculating durations, and finally formatting everything into a report. That approach made the whole process a lot easier to manage.

8.6 Code

```
1 # Makefile for PS7 (Python version)
```

```
2
```

```

3 .PHONY: all ps7 lint clean
4
5 # Create an executable-like wrapper for the Python script
6 all: ps7
7
8 # This generates a bash script that calls ps7.py
9 ps7:
10     echo "#!/bin/bash\npython3 ps7.py \$$1" > ps7
11     chmod +x ps7
12
13 # Lint with flake8
14 lint:
15     flake8 ps7.py || true
16
17 # Clean generated report files and wrapper script
18 clean:
19     rm -f ps7 *.rpt

```

```

1 # Copyright 2025 Lillyanna Yim
2
3 import sys
4 import re
5 from datetime import datetime
6
7 # Ensure correct number of command-line arguments are provided
8 if len(sys.argv) != 2:
9     print("Usage: python ps7.py <logfile>")
10    sys.exit(1)
11
12 # Get input log filename from command-line
13 log_filename = sys.argv[1]
14 # Create output report filename
15 report_filename = log_filename + ".rpt"
16
17 # Regular expression patterns to detect relevant log entries
18 start_pattern = re.compile(r'\(log\.c\.166\) server started') # Boot start
19 end_pattern = re.compile(r'oejs\.AbstractConnector:Started
    SelectChannelConnector@') # Boot complete
20 ts_start_pattern = re.compile(r'(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})') #
    Start timestamp
21 ts_end_pattern = re.compile(r'(\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2})\.\d{3})')
    # End timestamp with ms
22
23 # Tracking boots and state
24 boots = [] # List to hold each boot attempt
25 current_boot = None # Temporary storage for active boot
26 line_count = 0 # Count total lines read
27 completed_count = 0 # Count successful boots
28
29 # Read full log file into a list of lines
30 with open(log_filename, 'r') as file:
31     lines = file.readlines()
32
33 # Loop through each line and track boots
34 for i, line in enumerate(lines):
35     line_count += 1
36
37     # Detect a boot start event
38     if start_pattern.search(line):
39         # If previous boot was still in progress it's marked incomplete

```

```

40     if current_boot is not None:
41         boots.append({
42             "start_line": current_boot["start_line"],
43             "start_time": current_boot["start_time"],
44             "completed": False
45         })
46
47     # Extract timestamp and start tracking this boot
48     ts_match = ts_start_pattern.search(line)
49     if ts_match:
50         start_time = datetime.strptime(ts_match.group(1), "%Y-%m-%d %H:%M:%S")
51         current_boot = {
52             "start_line": i + 1,
53             "start_time": start_time
54         }
55
56     # Detect boot completion event that pairs with current boot
57     elif current_boot is not None and end_pattern.search(line):
58         ts_match = ts_end_pattern.search(line)
59         if ts_match:
60             end_time = datetime.strptime(ts_match.group(1), "%Y-%m-%d %H:%M:%S.%f")
61             # Calculate duration in milliseconds
62             duration_ms = int((end_time - current_boot["start_time"]).
total_seconds() * 1000)
63
64             # Record completed boot
65             boots.append({
66                 "start_line": current_boot["start_line"],
67                 "start_time": current_boot["start_time"],
68                 "end_line": i + 1,
69                 "end_time": end_time,
70                 "duration_ms": duration_ms,
71                 "completed": True
72             })
73
74             completed_count += 1
75             current_boot = None # Reset for next boot
76
77 # If boot was started but never completed (EOF) mark it incomplete
78 if current_boot is not None:
79     boots.append({
80         "start_line": current_boot["start_line"],
81         "start_time": current_boot["start_time"],
82         "completed": False
83     })
84
85 # Write formatted report to the output .rpt file
86 with open(report_filename, 'w') as f:
87     f.write("Device Boot Report\n\n")
88     f.write(f"InTouch log file: {log_filename}\n")
89     f.write(f"Lines Scanned: {line_count}\n\n")
90     f.write(f"Device boot count: initiated = {len(boots)}, completed: {
completed_count}\n\n")
91
92     # Loop through each boot and write its details
93     for boot in boots:
94         f.write("=== Device boot ===\n")

```

```

95     f.write(f"{boot['start_line']}({log_filename}): {boot['start_time']}
    Boot Start\n")
96
97     # Write completion line and duration if available
98     if boot["completed"]:
99         f.write(f"{boot['end_line']}({log_filename}): {boot['end_time']}.
    strftime('%Y-%m-%d %H:%M:%S')} Boot Completed\n")
100        f.write(f"\tBoot Time: {boot['duration_ms']}ms\n\n")
101    else:
102        # Indicate boot failure
103        f.write("**** Incomplete boot ****\n\n")
104
105 # Confirm to user that the report was created
106 print(f"Report written to: {report_filename}")

```

```

1 Device Boot Report
2
3 InTouch log file: device1_intouch.log
4 Lines Scanned: 443838
5
6 Device boot count: initiated = 6, completed: 6
7
8 === Device boot ===
9 435369(device1_intouch.log): 2014-03-25 19:11:59 Boot Start
10 435759(device1_intouch.log): 2014-03-25 19:15:02 Boot Completed
11     Boot Time: 183369ms
12
13 === Device boot ===
14 436500(device1_intouch.log): 2014-03-25 19:29:59 Boot Start
15 436859(device1_intouch.log): 2014-03-25 19:32:44 Boot Completed
16     Boot Time: 165036ms
17
18 === Device boot ===
19 440719(device1_intouch.log): 2014-03-25 22:01:46 Boot Start
20 440791(device1_intouch.log): 2014-03-25 22:04:27 Boot Completed
21     Boot Time: 161514ms
22
23 === Device boot ===
24 440866(device1_intouch.log): 2014-03-26 12:47:42 Boot Start
25 441216(device1_intouch.log): 2014-03-26 12:50:29 Boot Completed
26     Boot Time: 167824ms
27
28 === Device boot ===
29 442094(device1_intouch.log): 2014-03-26 20:41:34 Boot Start
30 442432(device1_intouch.log): 2014-03-26 20:44:13 Boot Completed
31     Boot Time: 159235ms
32
33 === Device boot ===
34 443073(device1_intouch.log): 2014-03-27 14:09:01 Boot Start
35 443411(device1_intouch.log): 2014-03-27 14:11:42 Boot Completed
36     Boot Time: 161500ms

```

```

1 Device Boot Report
2
3 InTouch log file: device2_intouch.log
4 Lines Scanned: 538352
5
6 Device boot count: initiated = 1, completed: 1
7
8 === Device boot ===

```



```
9 498921(device2_intouch.log): 2014-03-11 15:42:26 Boot Start
10 499030(device2_intouch.log): 2014-03-11 15:45:08 Boot Completed
11     Boot Time: 162023ms
```

```
1 Device Boot Report
2
3 InTouch log file: device3_intouch.log
4 Lines Scanned: 41855
5
6 Device boot count: initiated = 25, completed: 10
7
8 === Device boot ===
9 31063(device3_intouch.log): 2014-01-26 09:55:07 Boot Start
10 31176(device3_intouch.log): 2014-01-26 09:58:04 Boot Completed
11     Boot Time: 177362ms
12
13 === Device boot ===
14 31274(device3_intouch.log): 2014-01-26 12:15:18 Boot Start
15 **** Incomplete boot ****
16
17 === Device boot ===
18 31293(device3_intouch.log): 2014-01-26 14:02:39 Boot Start
19 31401(device3_intouch.log): 2014-01-26 14:05:24 Boot Completed
20     Boot Time: 165465ms
21
22 === Device boot ===
23 32623(device3_intouch.log): 2014-01-27 12:27:55 Boot Start
24 **** Incomplete boot ****
25
26 === Device boot ===
27 32641(device3_intouch.log): 2014-01-27 12:30:23 Boot Start
28 **** Incomplete boot ****
29
30 === Device boot ===
31 32656(device3_intouch.log): 2014-01-27 12:32:51 Boot Start
32 **** Incomplete boot ****
33
34 === Device boot ===
35 32674(device3_intouch.log): 2014-01-27 12:35:19 Boot Start
36 **** Incomplete boot ****
37
38 === Device boot ===
39 32693(device3_intouch.log): 2014-01-27 14:02:38 Boot Start
40 32801(device3_intouch.log): 2014-01-27 14:05:21 Boot Completed
41     Boot Time: 163193ms
42
43 === Device boot ===
44 33709(device3_intouch.log): 2014-01-28 12:44:17 Boot Start
45 **** Incomplete boot ****
46
47 === Device boot ===
48 33725(device3_intouch.log): 2014-01-28 14:02:33 Boot Start
49 33833(device3_intouch.log): 2014-01-28 14:05:15 Boot Completed
50     Boot Time: 162905ms
51
52 === Device boot ===
53 34594(device3_intouch.log): 2014-01-29 12:43:07 Boot Start
54 **** Incomplete boot ****
55
56 === Device boot ===
```



```

57 34613(device3_intouch.log): 2014-01-29 14:02:35 Boot Start
58 34721(device3_intouch.log): 2014-01-29 14:05:19 Boot Completed
59     Boot Time: 164715ms
60
61 === Device boot ===
62 37428(device3_intouch.log): 2014-01-30 12:43:05 Boot Start
63 **** Incomplete boot ****
64
65 === Device boot ===
66 37447(device3_intouch.log): 2014-01-30 14:02:40 Boot Start
67 37555(device3_intouch.log): 2014-01-30 14:05:22 Boot Completed
68     Boot Time: 162876ms
69
70 === Device boot ===
71 38258(device3_intouch.log): 2014-01-31 14:02:33 Boot Start
72 38366(device3_intouch.log): 2014-01-31 14:05:16 Boot Completed
73     Boot Time: 163257ms
74
75 === Device boot ===
76 39150(device3_intouch.log): 2014-02-01 12:39:38 Boot Start
77 **** Incomplete boot ****
78
79 === Device boot ===
80 39166(device3_intouch.log): 2014-02-01 12:42:07 Boot Start
81 **** Incomplete boot ****
82
83 === Device boot ===
84 39182(device3_intouch.log): 2014-02-01 14:02:32 Boot Start
85 39290(device3_intouch.log): 2014-02-01 14:05:16 Boot Completed
86     Boot Time: 164090ms
87
88 === Device boot ===
89 40288(device3_intouch.log): 2014-02-02 14:02:39 Boot Start
90 40397(device3_intouch.log): 2014-02-02 14:05:31 Boot Completed
91     Boot Time: 172252ms
92
93 === Device boot ===
94 41615(device3_intouch.log): 2014-02-03 12:35:55 Boot Start
95 **** Incomplete boot ****
96
97 === Device boot ===
98 41633(device3_intouch.log): 2014-02-03 12:38:22 Boot Start
99 **** Incomplete boot ****
100
101 === Device boot ===
102 41648(device3_intouch.log): 2014-02-03 12:40:48 Boot Start
103 **** Incomplete boot ****
104
105 === Device boot ===
106 41666(device3_intouch.log): 2014-02-03 12:43:17 Boot Start
107 **** Incomplete boot ****
108
109 === Device boot ===
110 41684(device3_intouch.log): 2014-02-03 12:45:46 Boot Start
111 **** Incomplete boot ****
112
113 === Device boot ===
114 41694(device3_intouch.log): 2014-02-03 14:02:34 Boot Start
115 41802(device3_intouch.log): 2014-02-03 14:05:18 Boot Completed

```

116	Boot Time: 164597ms
1	Device Boot Report
2	
3	InTouch log file: device4_intouch.log
4	Lines Scanned: 48793
5	
6	Device boot count: initiated = 15, completed: 15
7	
8	=== Device boot ===
9	4(device4_intouch.log): 2013-10-02 18:42:38 Boot Start
10	112(device4_intouch.log): 2013-10-02 18:45:23 Boot Completed
11	Boot Time: 165162ms
12	
13	=== Device boot ===
14	747(device4_intouch.log): 2013-10-03 12:23:21 Boot Start
15	855(device4_intouch.log): 2013-10-03 12:26:15 Boot Completed
16	Boot Time: 174017ms
17	
18	=== Device boot ===
19	1459(device4_intouch.log): 2013-10-04 16:20:03 Boot Start
20	1568(device4_intouch.log): 2013-10-04 16:23:06 Boot Completed
21	Boot Time: 183106ms
22	
23	=== Device boot ===
24	31848(device4_intouch.log): 2013-12-03 16:21:13 Boot Start
25	31956(device4_intouch.log): 2013-12-03 16:24:08 Boot Completed
26	Boot Time: 175443ms
27	
28	=== Device boot ===
29	32789(device4_intouch.log): 2013-12-04 21:50:27 Boot Start
30	33032(device4_intouch.log): 2013-12-04 21:52:57 Boot Completed
31	Boot Time: 150152ms
32	
33	=== Device boot ===
34	33145(device4_intouch.log): 2013-12-04 21:58:45 Boot Start
35	33390(device4_intouch.log): 2013-12-04 22:01:14 Boot Completed
36	Boot Time: 149024ms
37	
38	=== Device boot ===
39	33677(device4_intouch.log): 2013-12-04 22:21:03 Boot Start
40	33920(device4_intouch.log): 2013-12-04 22:23:31 Boot Completed
41	Boot Time: 148765ms
42	
43	=== Device boot ===
44	45295(device4_intouch.log): 2013-12-05 13:34:25 Boot Start
45	45538(device4_intouch.log): 2013-12-05 13:36:55 Boot Completed
46	Boot Time: 150752ms
47	
48	=== Device boot ===
49	45615(device4_intouch.log): 2013-12-05 14:12:25 Boot Start
50	45858(device4_intouch.log): 2013-12-05 14:14:53 Boot Completed
51	Boot Time: 148854ms
52	
53	=== Device boot ===
54	46117(device4_intouch.log): 2013-12-05 15:39:02 Boot Start
55	46353(device4_intouch.log): 2013-12-05 15:41:29 Boot Completed
56	Boot Time: 147230ms
57	
58	=== Device boot ===

```

59 46357(device4_intouch.log): 2013-12-05 20:20:24 Boot Start
60 46600(device4_intouch.log): 2013-12-05 20:22:54 Boot Completed
61     Boot Time: 150332ms
62
63 === Device boot ===
64 46792(device4_intouch.log): 2013-12-10 13:20:43 Boot Start
65 47035(device4_intouch.log): 2013-12-10 13:23:12 Boot Completed
66     Boot Time: 149552ms
67
68 === Device boot ===
69 47700(device4_intouch.log): 2013-12-10 19:40:58 Boot Start
70 47991(device4_intouch.log): 2013-12-10 19:43:55 Boot Completed
71     Boot Time: 177240ms
72
73 === Device boot ===
74 48100(device4_intouch.log): 2013-12-11 14:09:11 Boot Start
75 48341(device4_intouch.log): 2013-12-11 14:11:41 Boot Completed
76     Boot Time: 150568ms
77
78 === Device boot ===
79 48345(device4_intouch.log): 2013-12-11 14:17:49 Boot Start
80 48636(device4_intouch.log): 2013-12-11 14:20:46 Boot Completed
81     Boot Time: 177182ms

```

```

1  Device Boot Report
2
3  InTouch log file: device5_intouch.log
4  Lines Scanned: 41855
5
6  Device boot count: initiated = 25, completed: 10
7
8  === Device boot ===
9  31063(device5_intouch.log): 2014-01-26 09:55:07 Boot Start
10 31176(device5_intouch.log): 2014-01-26 09:58:04 Boot Completed
11     Boot Time: 177362ms
12
13 === Device boot ===
14 31274(device5_intouch.log): 2014-01-26 12:15:18 Boot Start
15 **** Incomplete boot ****
16
17 === Device boot ===
18 31293(device5_intouch.log): 2014-01-26 14:02:39 Boot Start
19 31401(device5_intouch.log): 2014-01-26 14:05:24 Boot Completed
20     Boot Time: 165465ms
21
22 === Device boot ===
23 32623(device5_intouch.log): 2014-01-27 12:27:55 Boot Start
24 **** Incomplete boot ****
25
26 === Device boot ===
27 32641(device5_intouch.log): 2014-01-27 12:30:23 Boot Start
28 **** Incomplete boot ****
29
30 === Device boot ===
31 32656(device5_intouch.log): 2014-01-27 12:32:51 Boot Start
32 **** Incomplete boot ****
33
34 === Device boot ===
35 32674(device5_intouch.log): 2014-01-27 12:35:19 Boot Start
36 **** Incomplete boot ****

```

```

37
38 === Device boot ===
39 32693(device5_intouch.log): 2014-01-27 14:02:38 Boot Start
40 32801(device5_intouch.log): 2014-01-27 14:05:21 Boot Completed
41     Boot Time: 163193ms
42
43 === Device boot ===
44 33709(device5_intouch.log): 2014-01-28 12:44:17 Boot Start
45 **** Incomplete boot ****
46
47 === Device boot ===
48 33725(device5_intouch.log): 2014-01-28 14:02:33 Boot Start
49 33833(device5_intouch.log): 2014-01-28 14:05:15 Boot Completed
50     Boot Time: 162905ms
51
52 === Device boot ===
53 34594(device5_intouch.log): 2014-01-29 12:43:07 Boot Start
54 **** Incomplete boot ****
55
56 === Device boot ===
57 34613(device5_intouch.log): 2014-01-29 14:02:35 Boot Start
58 34721(device5_intouch.log): 2014-01-29 14:05:19 Boot Completed
59     Boot Time: 164715ms
60
61 === Device boot ===
62 37428(device5_intouch.log): 2014-01-30 12:43:05 Boot Start
63 **** Incomplete boot ****
64
65 === Device boot ===
66 37447(device5_intouch.log): 2014-01-30 14:02:40 Boot Start
67 37555(device5_intouch.log): 2014-01-30 14:05:22 Boot Completed
68     Boot Time: 162876ms
69
70 === Device boot ===
71 38258(device5_intouch.log): 2014-01-31 14:02:33 Boot Start
72 38366(device5_intouch.log): 2014-01-31 14:05:16 Boot Completed
73     Boot Time: 163257ms
74
75 === Device boot ===
76 39150(device5_intouch.log): 2014-02-01 12:39:38 Boot Start
77 **** Incomplete boot ****
78
79 === Device boot ===
80 39166(device5_intouch.log): 2014-02-01 12:42:07 Boot Start
81 **** Incomplete boot ****
82
83 === Device boot ===
84 39182(device5_intouch.log): 2014-02-01 14:02:32 Boot Start
85 39290(device5_intouch.log): 2014-02-01 14:05:16 Boot Completed
86     Boot Time: 164090ms
87
88 === Device boot ===
89 40288(device5_intouch.log): 2014-02-02 14:02:39 Boot Start
90 40397(device5_intouch.log): 2014-02-02 14:05:31 Boot Completed
91     Boot Time: 172252ms
92
93 === Device boot ===
94 41615(device5_intouch.log): 2014-02-03 12:35:55 Boot Start
95 **** Incomplete boot ****

```

```

96
97 === Device boot ===
98 41633(device5_intouch.log): 2014-02-03 12:38:22 Boot Start
99 **** Incomplete boot ****
100
101 === Device boot ===
102 41648(device5_intouch.log): 2014-02-03 12:40:48 Boot Start
103 **** Incomplete boot ****
104
105 === Device boot ===
106 41666(device5_intouch.log): 2014-02-03 12:43:17 Boot Start
107 **** Incomplete boot ****
108
109 === Device boot ===
110 41684(device5_intouch.log): 2014-02-03 12:45:46 Boot Start
111 **** Incomplete boot ****
112
113 === Device boot ===
114 41694(device5_intouch.log): 2014-02-03 14:02:34 Boot Start
115 41802(device5_intouch.log): 2014-02-03 14:05:18 Boot Completed
116     Boot Time: 164597ms

```

```

1 Device Boot Report
2
3 InTouch log file: device6_intouch.log
4 Lines Scanned: 90671
5
6 Device boot count: initiated = 14, completed: 12
7
8 === Device boot ===
9 2(device6_intouch.log): 2014-04-03 20:27:48 Boot Start
10 161(device6_intouch.log): 2014-04-03 20:31:01 Boot Completed
11     Boot Time: 193087ms
12
13 === Device boot ===
14 82079(device6_intouch.log): 2014-04-09 14:51:15 Boot Start
15 82303(device6_intouch.log): 2014-04-09 14:54:39 Boot Completed
16     Boot Time: 204150ms
17
18 === Device boot ===
19 85398(device6_intouch.log): 2014-04-10 18:13:13 Boot Start
20 85564(device6_intouch.log): 2014-04-10 18:16:37 Boot Completed
21     Boot Time: 204082ms
22
23 === Device boot ===
24 85957(device6_intouch.log): 2014-04-10 19:11:05 Boot Start
25 86123(device6_intouch.log): 2014-04-10 19:14:24 Boot Completed
26     Boot Time: 199863ms
27
28 === Device boot ===
29 86127(device6_intouch.log): 2014-04-10 19:18:36 Boot Start
30 86293(device6_intouch.log): 2014-04-10 19:21:56 Boot Completed
31     Boot Time: 200771ms
32
33 === Device boot ===
34 86568(device6_intouch.log): 2014-04-10 19:32:16 Boot Start
35 86732(device6_intouch.log): 2014-04-10 19:35:36 Boot Completed
36     Boot Time: 200317ms
37
38 === Device boot ===

```

```
39 86750(device6_intouch.log): 2014-04-10 20:06:27 Boot Start
40 86821(device6_intouch.log): 2014-04-10 20:09:07 Boot Completed
41     Boot Time: 160793ms
42
43 === Device boot ===
44 86939(device6_intouch.log): 2014-04-11 00:15:56 Boot Start
45 87111(device6_intouch.log): 2014-04-11 00:18:49 Boot Completed
46     Boot Time: 173131ms
47
48 === Device boot ===
49 87116(device6_intouch.log): 2014-04-11 13:28:25 Boot Start
50 87286(device6_intouch.log): 2014-04-11 13:31:12 Boot Completed
51     Boot Time: 167767ms
52
53 === Device boot ===
54 87836(device6_intouch.log): 2014-04-11 13:58:02 Boot Start
55 88009(device6_intouch.log): 2014-04-11 14:00:49 Boot Completed
56     Boot Time: 167095ms
57
58 === Device boot ===
59 88983(device6_intouch.log): 2014-04-11 14:23:42 Boot Start
60 89155(device6_intouch.log): 2014-04-11 14:26:31 Boot Completed
61     Boot Time: 169377ms
62
63 === Device boot ===
64 90112(device6_intouch.log): 2014-04-14 12:13:59 Boot Start
65 **** Incomplete boot ****
66
67 === Device boot ===
68 90135(device6_intouch.log): 2014-04-14 12:16:13 Boot Start
69 **** Incomplete boot ****
70
71 === Device boot ===
72 90176(device6_intouch.log): 2014-04-14 12:18:44 Boot Start
73 90307(device6_intouch.log): 2014-04-14 12:21:25 Boot Completed
74     Boot Time: 161493ms
```